

4

MARKOV CHAIN MONTE CARLO

In this chapter, we introduce Markov chains and then Markov Chain Monte Carlo (MCMC) methods. These methods are at the heart of Bayesian statistics, generative modelling, statistical physics, and many other fields. We will introduce the Metropolis-Hastings algorithm and then introduce the celebrated Gibbs sampler and, if time permits, some others.



In this chapter, we introduce a new sampling methodology using Markov chains to generate samples from complicated target distributions. This is a powerful and widely used idea in statistics and machine learning. The idea is to design Markov chains with prescribed stationary distributions so that, after they mix, their states are distributed according to the target.

4.1 DISCRETE STATE SPACE MARKOV CHAINS

A good setting for an introduction to Markov chains is the discrete space setting. In this setting, we have a finite set of states $X = \{1, 2, \dots, d\}$, where the cardinality of X is finite. We first define the Markov chain in this context.

Definition 4.1 (Markov chain). *A discrete-time, discrete-state Markov chain on X is a stochastic process $(X_n)_{n \geq 0}$ with values in X such that for all $n \geq 0$ and all $x_0, \dots, x_{n+1} \in X$ with $\mathbb{P}(X_0 = x_0, \dots, X_n = x_n) > 0$,*

$$\mathbb{P}(X_{n+1} = x_{n+1} \mid X_0 = x_0, \dots, X_n = x_n) = \mathbb{P}(X_{n+1} = x_{n+1} \mid X_n = x_n).$$

In other words, a Markov chain is a sequence of random variables such that the state at time n is conditionally independent of all earlier states given the state at time $n - 1$. This captures many real-world systems, whose evolution is determined by the current state and an evolution law.

An important object in the study of Markov chains is the transition matrix (or kernel in the continuous space case). It encodes the evolution of the chain and determines its properties. The transition matrix is defined as follows.

Definition 4.2 (Transition matrix). *For a discrete-time Markov chain $(X_n)_{n \geq 0}$ on $\mathsf{X} = \{1, \dots, d\}$, the transition matrix is the $d \times d$ matrix M with entries*

$$M_{ij} = \mathbb{P}(X_{n+1} = j \mid X_n = i), \quad \text{for all } i, j \in \mathsf{X} \text{ and all } n \geq 0.$$

In particular, $M_{ij} \geq 0$ for all i, j and $\sum_{j=1}^d M_{ij} = 1$ for each i .

A usual way to depict Markov chains is the following conditional independence structure which sums the structure of the Markov chain up. We note that we will only consider the case

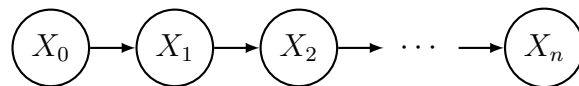


Figure 4.1: A Markov chain with n states.

where the transition matrix is time-homogeneous, i.e., the transition matrix is the same for all times. We can see then that a Markov chain's behaviour is completely determined by its initial distribution and the transition matrix. We will denote the initial distribution of the chain as p_0 and note that this is a discrete distribution over the state space X (in this case)¹. The transition matrix M is a matrix of size $d \times d$ where $d = |\mathsf{X}|$.

$$M = \begin{bmatrix} M_{11} & M_{12} & \cdots & M_{1d} \\ M_{21} & M_{22} & \cdots & M_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ M_{d1} & M_{d2} & \cdots & M_{dd} \end{bmatrix}.$$

We note that this matrix is stochastic, i.e. each row sums to 1:

$$\sum_{j=1}^d M_{ij} = 1,$$

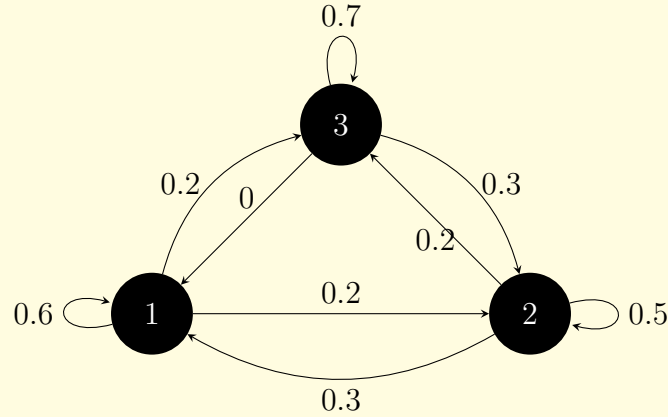
since $M_{ij} = \mathbb{P}(X_{n+1} = j \mid X_n = i)$ and

$$\sum_{j=1}^d \mathbb{P}(X_{n+1} = j \mid X_n = i) = 1.$$

Next we will consider an example.

¹When we move to continuous spaces, we will use the same notation for densities.

Example 4.1 (Discrete space Markov chain). Consider the state transition diagram



of a Markov chain. Write out the transition matrix of this chain and describe the simulation procedure.

Solution. We can write the transition matrix as

$$M = \begin{bmatrix} 0.6 & 0.2 & 0.2 \\ 0.3 & 0.5 & 0.2 \\ 0 & 0.3 & 0.7 \end{bmatrix}, \quad \text{where } X = \{1, 2, 3\}.$$

The state transition diagram of this matrix can be described as follows.

This Markov chain can be simulated using the following idea. Given the above diagram, we can denote its transition matrix as a table:

M	$X_t = 1$	$X_t = 2$	$X_t = 3$
$X_{t-1} = 1$	0.6	0.2	0.2
$X_{t-1} = 2$	0.3	0.5	0.2
$X_{t-1} = 3$	0	0.3	0.7

Given $X_0 = 1$, how to simulate this chain? This boils down to just selecting the correct row from this matrix and then sampling using the discrete distribution given by that row. For example, if we sample from the first row, we get $X_1 = 1$ with probability 0.6, $X_1 = 2$ with probability 0.2 and $X_1 = 3$ with probability 0.2. We can then repeat this process for X_2 and so on. This is a simple way to simulate a Markov chain. The precise sampler is given below.

$$X_t | X_{t-1} = x_{t-1} \sim \text{Discrete}(M_{x_{t-1}, \cdot}),$$

where the notation $M_{x_{t-1}, \cdot}$ denotes the x_{t-1} th row of the transition matrix M (where $x_{t-1} \in \{1, 2, 3\}$ naturally).

Markov chains can be seen everywhere in the real world, hence we will skip their applications. But one exciting example can be provided from the field of language modelling. Let us describe a character-level Markov model of English.

Example 4.2 (Character-level Markov model of English). Consider the task of modelling an English language text as a Markov model. Describe a possible way of doing this on a character-level.

Solution. We can model the English language as a Markov chain on a character-level. In this case, we can define the state space as $X = \{a, b, \dots, z, .\}$ (where for simplicity the character dot models the space character and no other special characters are considered). We can then define the transition matrix as follows. Let x_{t-1} be the previous character and x_t be the current character. Of course, writing out the transition matrix by hand is an impossible task, hence we can *learn* it from data. For this, we simply choose a large corpus of English text and count the number of times a character is followed by another character. We have provided an example code for this in our online companion. By estimating the transition matrix from data, we can then simulate English text by sampling from this Markov chain. For example, one simulated text reads as

`g.l.se.t.bin.s.lese.ry..wolked.t.hered.e.ly.hr.impefatrt.mofe.mouroreand`
and goes on (see lecture for a longer simulation). We can see that while this captures very vague structure, it is not a good model of English. However, this character-level model can still be useful for some applications. Because we can use this model to estimate the probability of a given text. For example, we can compute

$$\mathbb{P}(\text{the quick brown fox jumps over the lazy dog})$$

by breaking down this text into characters and ‘reading’ the transition matrix. This is a very simple example of a Markov chain model of English.

Let us look at a more intelligent way of using Markov chains to model language.

Example 4.3 (Word-level Markov model of English). Consider the task of modelling an English language text as a Markov model. Describe a possible way of doing this on a word-level.

Solution. We can model the English language as a Markov chain on a word-level. As you can see, there is no single state-space for this (as we can expand our word list (state-space) possibly indefinitely). A good idea is to pick a book or a large text corpus and count all possible word transitions. Note that modern language models do *not* use words, but rather sub-word units, called tokens. Here we will stick to words and we will pick the book ‘A Study in Scarlet’ by Sir Arthur Conan Doyle. We have provided an example code for this in our online companion. By estimating the transition matrix from data, we can then simulate English text by sampling from this Markov chain. For example, one simulated text reads as

Sherlock Holmes, "because you will place where you remember, in his cudgel, only one in summer are grey with a vague shadowy terrors which hung over my hand, and eyeing the strange contrast to the name a walking in time before twelve, you are."

While this is much more intelligible and seems to follow some English rules, it is nothing but a simulation from a Markov chain given a transition matrix as estimated in our online companion. Modern language models, however, do not use Markov models either – their structure is much more complex and they are based on neural networks.

We can also compute the n -step transition matrix. Define $M^{(n)}$ by its entries

$$M_{ij}^{(n)} = \mathbb{P}(X_n = j \mid X_0 = i), \quad i, j \in \mathbf{X},$$

where $M^{(n)}$ is a $d \times d$ matrix. By the law of total probability and the Markov property, for $n \geq 1$ we have

$$\begin{aligned} M_{ij}^{(n)} &= \mathbb{P}(X_n = j \mid X_0 = i) \\ &= \sum_{k \in \mathbf{X}} \mathbb{P}(X_n = j, X_1 = k \mid X_0 = i) \\ &= \sum_{k \in \mathbf{X}} \mathbb{P}(X_n = j \mid X_1 = k, X_0 = i) \mathbb{P}(X_1 = k \mid X_0 = i) \\ &= \sum_{k \in \mathbf{X}} \mathbb{P}(X_n = j \mid X_1 = k) \mathbb{P}(X_1 = k \mid X_0 = i) \\ &= \sum_{k \in \mathbf{X}} M_{ik} M_{kj}^{(n-1)} = (M M^{(n-1)})_{ij}. \end{aligned}$$

This last equality is precisely the definition of the (i, j) entry of the matrix product $M M^{(n-1)}$. Therefore, by induction on n , $M^{(n)} = M^n$, the n th power of the transition matrix. Note that we can compute in general the conditional distributions of the Markov chain by summing out the variables in the middle. For example, in order to compute $\mathbb{P}(X_{n+2} = x_{n+2} \mid X_n = x_n)$, we can write

$$\mathbb{P}(X_{n+2} = x_{n+2} \mid X_n = x_n) = \sum_{x_{n+1} \in \mathbf{X}} \mathbb{P}(X_{n+2} = x_{n+2} \mid X_{n+1} = x_{n+1}) \mathbb{P}(X_{n+1} = x_{n+1} \mid X_n = x_n).$$

This leads to the Chapman–Kolmogorov equations: for all $m, n \geq 0$ and $i, j \in \mathbf{X}$,

$$\begin{aligned} M_{ij}^{(m+n)} &= \mathbb{P}(X_{m+n} = j \mid X_0 = i) \\ &= \sum_{k \in \mathbf{X}} \mathbb{P}(X_{m+n} = j \mid X_n = k) \mathbb{P}(X_n = k \mid X_0 = i) \\ &= \sum_{k \in \mathbf{X}} M_{ik}^{(m)} M_{kj}^{(n)}. \end{aligned}$$

Therefore, in matrix form, $M^{(m+n)} = M^{(m)} M^{(n)}$.

It is also important to define the evolution of the chain. Note that we defined our initial distribution as p_0 and it is important to quantify how this distribution evolves over time. We denote the distribution at time n as p_n and write Then, the density of the chain at time n is given

by:

$$\begin{aligned}
p_n(i) &= \mathbb{P}(X_n = i) \\
&= \sum_k \mathbb{P}(X_n = i, X_{n-1} = k) \\
&= \sum_k \mathbb{P}(X_n = i | X_{n-1} = k) \mathbb{P}(X_{n-1} = k) \\
&= \sum_k M_{ki} p_{n-1}(k).
\end{aligned}$$

This implies that

$$p_n = p_{n-1} M.$$

Therefore,

$$p_n = p_0 M^n.$$

These are important equations, which will have corresponding equations in the continuous case (however, they will be integrals).

Since we have expressed our interest in Markov chains because of their potential utility in sampling, we will now discuss the properties we need to ensure that we can use Markov chains for sampling. In short, we need Markov chains that have (i) invariant distributions, (ii) their convergence to invariant distributions are ensured, (iii) the invariant distribution is unique. We will now discuss the properties we need to ensure these in detail.

4.1.1 IRREDUCIBILITY

The first property we need to ensure is that the Markov chain is irreducible. This means that there is a path from any state to any other state. To be precise, let $x, x' \in \mathbf{X}$ be any two states. We write $x \rightsquigarrow x'$ if there is a path from x to x' :

$$\exists n > 0, \text{ s.t. }, \mathbb{P}(X_n = x' | X_0 = x) > 0.$$

If $x \rightsquigarrow x'$ and $x' \rightsquigarrow x$, then we say that x and x' *communicate*. We then define the *communication class* $C \subset \mathbf{X}$ which is a set of states such that $x \in C$ and $x' \in C$ if and only if $x \rightsquigarrow x'$ and $x' \rightsquigarrow x$. A chain is irreducible if $\mathbf{X}$ is a single communication class. This simply means that there is a positive probability of moving around to every other state. This makes sense as without ensuring this, we won't be sampling from the full support.

4.1.2 RECURRENCE AND TRANSIENCE

We now define the notion of recurrence and transience. A state $i \in \mathbf{X}$ is *recurrent* if there is a positive probability of returning to i . In order to see define this, consider the return time

$$\tau_i = \inf\{n \geq 1 : X_n = i\}.$$

We say that the state i is recurrent if

$$\mathbb{P}(\tau_i < \infty | X_0 = i) = 1.$$

In other words, the probability of waiting time being finite is 1. If a chain is not recurrent, it is said to be transient. We can also further define the *positive recurrence* which is a slightly stronger (better) condition. We say that i is positively recurrent if

$$\mathbb{E}[\tau_i | X_0 = i] < \infty.$$

This means that the expected waiting time is finite. If a chain is recurrent but not positive recurrent, then it is called null recurrent.

4.1.3 INVARIANT DISTRIBUTIONS

In the discrete time case, a distribution p_* is called invariant if

$$p_* = p_* M.$$

This means that the chain is reach stationarity, i.e., evolving further (via M) does not change the distribution. We have then the following theorem (Yildirim, 2017).

Theorem 4.1. *If M is irreducible, then M has a unique invariant distribution if and only if it is positive recurrent.*

This is encouraging however for actual convergence of the chain to this distribution, we will need more conditions.

4.1.4 REVERSIBILITY AND DETAILED BALANCE

We define the detailed balance condition as

$$p_*(i)M_{ij} = p_*(j)M_{ji}.$$

This trivially implies that $p_* = p_* M$, hence the invariance of p_* . We will have a more detailed discussion of this condition in the continuous state space case.

4.1.5 CONVERGENCE TO INVARIANT DISTRIBUTION

Finally, we need the ergodicity condition to ensure that the chain converges to the invariant distribution. For this, we require the chain to be aperiodic, which is defined as follows. A state i is called aperiodic if

$$\{n > 0 : \mathbb{P}(X_{n+1} = i | X_1 = i) > 0\}$$

has no common divisor other than 1. A Markov chain is called aperiodic if all states are aperiodic. An irreducible Markov chain is called ergodic if it is positive recurrent and aperiodic. If $(X_n)_{n \in \mathbb{N}}$ is an ergodic Markov chain with initial p_0 and p_* as its invariant distribution, then

$$\lim_{n \rightarrow \infty} p_n(i) = p_*(i).$$

Moreover, for $i, j \in X$

$$\lim_{n \rightarrow \infty} \mathbb{P}(X_n = i | X_1 = j) = p_*(i).$$

In other words, the chain will converge to its invariant distribution from every state.

4.2 CONTINUOUS STATE SPACE MARKOV CHAINS

Our main interest is in the continuous case. However, it is important to understand the definitions above – as we will not go into analogous definitions in the continuous case. The reason for this is that, in continuous cases, the individual states have zero probability (i.e. a point has zero probability) and all the notions above are defined using sets and measure theoretic concepts. We focus on simulation methods within this course, therefore, we will not go into reviewing this material. A couple of very good books for this are [Douc et al. \(2018\)](#) and [Douc et al. \(2013\)](#).

Let X be an uncountable set from now on, e.g., $X = \mathbb{R}$ or $X = \mathbb{R}^d$. We denote the initial density as $p_0(x)$ as usual and the marginal density of the chain at time n as $p_n(x)$. A (time-homogeneous) Markov transition kernel is a mapping $K(\cdot|\cdot)$ such that for each $x' \in X$, $A \mapsto K(A|x')$ is a probability measure on X , and for each measurable $A \subseteq X$, $x' \mapsto K(A|x')$ is measurable. When $K(\cdot|x')$ admits a density with respect to a reference measure dx , we denote this density using the same letter $K(x|x')$, so that $K(A|x') = \int_A K(x|x') dx$.

We can write the Markov property in this case as follows. For any measurable A

$$\mathbb{P}(X_n \in A | X_1 = x_1, \dots, X_{n-1} = x_{n-1}) = \mathbb{P}(X_n \in A | X_{n-1} = x_{n-1}).$$

This implies that if we write down the joint distribution of $X_{1:n}$, then the following factorisation holds

$$p(x_0, \dots, x_n) = p_0(x_0) \prod_{k=1}^n K(x_k | x_{k-1}).$$

Similarly to the discrete case, we will assume that the density is time-homogeneous (i.e. same for every n). Note that the transition density is a density in its first variable, i.e.,

$$\int_X K(x_n | x_{n-1}) dx_n = 1.$$

It is a function of x_{n-1} otherwise. We give an example of a continuous state-space Markov chain in what follows.

Example 4.4 (Simulation of a Markov process). Consider the following Markov chain with $X_0 = 0$

$$X_n | X_{n-1} = x_{n-1} \sim \mathcal{N}(x_n; ax_{n-1}, 1), \tag{4.1}$$

with $0 < a < 1$. Describe the simulation procedure for this chain in terms of a recursion.

Solution. We can simulate this chain by

$$\begin{aligned} X_1 &\sim \mathcal{N}(0, 1) \\ X_2 &\sim \mathcal{N}(aX_1, 1) \\ X_3 &\sim \mathcal{N}(aX_2, 1) \\ &\vdots \\ X_n &\sim \mathcal{N}(aX_{n-1}, 1). \end{aligned}$$

How to do this? We also note that Eq. (4.1) can also be expressed as

$$X_n = aX_{n-1} + \epsilon_n$$

where $\epsilon_n \sim \mathcal{N}(0, 1)$. This is also called AR(1) process. From the last equation, it must be clear how to simulate this as you only need a for loop and samples from $\mathcal{N}(0, 1)$.

Exercise 4.1. Consider weights

$$w_1 = 0.2993 \quad \text{and} \quad w_2 = 0.7007,$$

and the following matrices A_1, A_2 and vectors b_1, b_2

$$\begin{aligned} A_1 &= \begin{bmatrix} 0.4 & -0.3733 \\ 0.06 & 0.6 \end{bmatrix} & \text{and} & b_1 = \begin{bmatrix} 0.3533 \\ 0.0 \end{bmatrix} \\ A_2 &= \begin{bmatrix} -0.8 & -0.1867 \\ 0.1371 & 0.8 \end{bmatrix} & \text{and} & b_2 = \begin{bmatrix} 1.1 \\ 0.1 \end{bmatrix} \end{aligned}$$

Now we will define a Markov chain on $\mathbb{R}^2$ and we will denote our chain with $(x_k)_{k \geq 0}$. For this, consider two deterministic functions:

$$\begin{aligned} f_1(x) &= A_1x + b_1 \\ f_2(x) &= A_2x + b_2. \end{aligned}$$

Simulate the following Markov process

$$\begin{aligned} i_n &\sim \text{Discrete}(w_1, w_2) \\ x_{n+1} &= f_{i_n}(x_n) \end{aligned}$$

for $1 \leq n \leq N$ where $N = 10000$. You can use $x_0 = [0, 0]^\top$. Plot a scatter plot of the Markov chain. For a pretty plot, you can use the following plotting function

```
1 plt.scatter(x[0, 20:k], x[1, 20:k], s=0.1, color = [0.8, 0, 0])
2 plt.gca().spines['top'].set_visible(False)
3 plt.gca().spines['right'].set_visible(False)
4 plt.gca().spines['bottom'].set_visible(False)
5 plt.gca().spines['left'].set_visible(False)
6 plt.gca().set_xticks([])
7 plt.gca().set_yticks([])
8 plt.gca().set_xlim(0, 1.05)
9 plt.gca().set_ylim(0, 1)
10 plt.show()
```

As you can see, I chose burnin as 20 iterations here. Be careful about matrix products in Python: **A*x is not performing matrix product**, you should use **A@x** where A is a 2×2 matrix and x is a 2×1 vector. Try to simulate from this system until you see a nice picture.

Similar to the discrete case, we can define the distribution of X_n given a past variable X_{n-k} by integrating out the variables in between. It is important to note that X_n is *conditionally* independent of past variables given X_{n-1} (the Markov property), i.e., $p(x_n|x_{0:n-1}) = p(x_n|x_{n-1}) = K(x_n|x_{n-1})$. Otherwise, we can write down the densities as

$$p(x_n|x_{n-k}) = \int_{\mathbb{X}} \cdots \int_{\mathbb{X}} K(x_n|x_{n-1})K(x_{n-1}|x_{n-2}) \cdots K(x_{n-k+1}|x_{n-k}) dx_{n-1} \cdots dx_{n-k+1}.$$

In other words, $p(x_n|x_{n-k}) = K^{(k)}(x_n|x_{n-k})$.

We define the m -step transition kernel as

$$K^{(m)}(x_{m+n}|x_n) = \int_{\mathbb{X}} \cdots \int_{\mathbb{X}} K(x_{m+n}|x_{m+n-1}) \cdots K(x_{n+1}|x_n) dx_{m+n-1} \cdots dx_{n+1}.$$

We now provide the definition of invariance in this context, w.r.t. to the transition kernel.

Definition 4.3 (K -invariance). *A density $p_\star$ on $\mathbb{X}$ is called K -invariant if, for $x \in \mathbb{X}$,*

$$p_\star(x) = \int_{\mathbb{X}} K(x|x') p_\star(x') dx'. \quad (4.2)$$

It can be seen that $p_\star$ being invariant means that the kernel operating on $p_\star$ results in the same distribution $p_\star$ (the integral against the kernel can be seen as a transformation, similar to the matrix product in the discrete case). Finally, we get to the detailed balance condition.

Definition 4.4 (Detailed balance). *A transition kernel K is said to satisfy detailed balance if*

$$K(x'|x)p_\star(x) = K(x|x')p_\star(x'). \quad (4.3)$$

Remark 4.1. It is also important to note the more general version of the detailed balance written using the integral form. In general, the transition kernel is a probability measure w.r.t. first argument so the above relationship can be written as

$$\int f(x, x') K(x'|x) p_\star(x) dx' dx = \int f(x, x') K(x|x') p_\star(x') dx' dx,$$

for f defined on $\mathbb{X} \times \mathbb{X}$.

We note that this is a sufficient condition for stationarity of $p_\star$.

Proposition 4.1 (Detailed balance implies stationarity). *If K satisfies detailed balance, then p_* is the invariant distribution.*

Proof. The proof is a one-liner:

$$\int p_*(x)K(x'|x)dx' = \int p_*(x')K(x|x')dx',$$

which is just integrating both sides after writing the detailed balance condition. The lhs of this equation is $p_*(x)$ since $K(x'|x)$ integrates to 1 which leaves us with the definition of K -invariance as given in (4.2). $\square$

Remark 4.2 (Convergence). Convergence to an invariant density requires additional regularity. A standard set of sufficient conditions (see, e.g., Douc et al. (2018)) is that the chain be ϕ -irreducible (every set of positive reference measure is eventually reachable), aperiodic, and Harris recurrent. Under these assumptions the invariant density, if it exists, is unique and for any initial density p_0 ,

$$p_n(x) \rightarrow p_*(x) \quad \text{in total variation as } n \rightarrow \infty.$$

Stronger drift or minorisation conditions yield quantitative rates (geometric ergodicity), but the qualitative message mirrors the discrete case: irreducibility, aperiodicity, and recurrence deliver convergence to stationarity.

Let us see an example of a continuous space Markov chain (or rather go back to AR(1) example).

Exercise 4.2. Consider the real-valued Markov kernel

$$K(x_n|x_{n-1}) = \mathcal{N}(x_n; ax_{n-1}, 1).$$

Show that this kernel satisfies the detailed balance condition w.r.t.

$$p_*(x) = \mathcal{N}\left(x; 0, \frac{1}{1-a^2}\right).$$

Exercise 4.3. Show that the kernel in the previous exercise satisfies

$$p_*(x) = \lim_{n \rightarrow \infty} K^{(n)}(x|x').$$

In other words, regardless from where we start, the kernel would converge to the stationary distribution.

Hint: Start from writing the recursions $x_{n+1} = ax_n + \epsilon_n$ where $\epsilon_n \sim \mathcal{N}(0, 1)$ and $x_0 = x'$. Try to first derive the expression of x_{n+1} in terms of x_0 . Compute the mean and the variance of x_{n+1} , then take the limit $n \rightarrow \infty$.

Remark 4.3. Consider the multivariate extension of the above AR(1) process:

$$X_n = AX_{n-1} + b + \Sigma^{1/2}\epsilon_n,$$

where $X_n, \epsilon_n, b \in \mathbb{R}^d$, $\epsilon_n \sim \mathcal{N}(0, I_d)$, $A \in \mathbb{R}^{d \times d}$ with spectral radius $\rho(A) < 1$, and $\Sigma \in \mathbb{R}^{d \times d}$ is a positive definite matrix. The stationary distribution is given by

$$p_\star(x) = \mathcal{N}(x; \mu, \Gamma),$$

where

$$\begin{aligned} \mu &= (I_d - A)^{-1}b, \\ \Gamma &= \sum_{k=0}^{\infty} A^k \Sigma (A^\top)^k. \end{aligned}$$

Alternatively, you can also show for Γ that it is the solution of the following Lyapunov equation:

$$\Gamma = A\Gamma A^\top + \Sigma.$$

We have now almost everything we need to move on to discuss Metropolis-Hastings method.

4.3 METROPOLIS-HASTINGS ALGORITHM

We finally have all the ingredients to define the celebrated Metropolis-Hastings algorithm. We will not need more technicalities in defining it.

The Metropolis-Hastings (MH) algorithm is a remarkable method which allows us to define transition kernels (defined implicitly via the algorithm) where the detailed balance is satisfied w.r.t. any $p_\star$ we wish to sample from. I call this *remarkable* because it rids us of the need of designing Markov kernels for specific probability distributions and provides a generic way to design samplers that will target any measure we want. The algorithm relies on the idea of using *local* proposals $q(x'|x)$ and accepting them with a certain acceptance ratio. The acceptance ratio is designed so that the resulting samples $X_1, \dots, X_n$ from the method form a Markov chain that leaves $p_\star$ invariant. We will provide the algorithm below, as seen from Algorithm 9. Note, as mentioned in the lecture, the last step of the method: When a sample is rejected, we do not sample again – we set $X_n = X_{n-1}$ and continue sampling the next sample. This means that, if the rejection rate is high, there will be a lot of duplicated samples and this is the expected behaviour.

Algorithm 9 Pseudocode for Metropolis Hastings method

- 1: Input: The number of samples N , and starting point X_0 .
- 2: **for** $n = 1, \dots, N$ **do**
- 3: Propose (sample): $X' \sim q(x'|X_{n-1})$
- 4: Accept the sample X' with probability

$$\alpha(X_{n-1}, X') = \min \left\{ 1, \frac{p_*(X')q(X_{n-1}|X')}{p_*(X_{n-1})q(X'|X_{n-1})} \right\}.$$

- 5: Otherwise reject the sample and set $X_n = X_{n-1}$.
 - 6: **end for**
 - 7: Discard first burnin samples and return the remaining samples.
-

Another important note is about the burnin period. Any Markov chain started at a random point will take some time to reach stationarity (the whole magic is to be able to make them converge faster). Therefore, we discard the first burnin samples and only return the remaining ones. This is a common practice in MCMC methods.

We define the acceptance ratio as

$$r(x, x') = \frac{p_*(x')q(x|x')}{p_*(x)q(x'|x)}. \quad (4.4)$$

We also note that in the practical algorithm, one does not need to implement the min operation. For accepting with a certain probability (like in the rejection sampling), we draw $U \sim \text{Unif}(0, 1)$ and check if $U \leq \alpha(X_{n-1}, X')$. However, if the ratio $r(X_{n-1}, X')$ is greater than 1, this sample is always going to be accepted anyway. The min operation is important however for theoretical properties of the kernel to hold.

As we mentioned above, the algorithm provides us with an implicit kernel $K(x_n|x_{n-1})$ – if you think about it, it is just a way to get X_n given X_{n-1} . The specific structure of the algorithm, however, ensures that we leave the right kind of distribution invariant – i.e. p_* – that is our target measure. We elucidate this in the following proposition.

Proposition 4.2 (Metropolis-Hastings satisfies detailed balance). *The Metropolis-Hastings algorithm satisfies detailed balance w.r.t. p_* , i.e.,*

$$p_*(x)K(x'|x) = p_*(x')K(x|x'),$$

where K is the kernel defined by the MH algorithm.

Proof. We first define the kernel induced by the MH algorithm. This can be seen by inspecting the algorithm - and we write it intuitively as (by writing Dirac functions instead of measures)

$$K(x'|x) = \alpha(x, x')q(x'|x) + (1 - \alpha(x, x'))\delta_x(x'),$$

where δ_x is the Dirac delta function and

$$a(x) = \int_{\mathbf{X}} \alpha(x, x') q(x'|x) dx',$$

is the probability of accepting a sample (hence $1 - a(x)$ is the probability of rejecting a new sample while at point x). See Sec. 2.3.1 of [Douc et al. \(2018\)](#) for a rigorous derivation. Given this, to check detailed balance, we use Remark 4.1 and write for a function f

$$\begin{aligned} \int f(x, x') p_{\star}(x) K(x'|x) dx dx' &= \int f(x, x') p_{\star}(x) q(x'|x) \alpha(x, x') dx dx' \\ &\quad + \int f(x, x') p_{\star}(x) (1 - a(x)) \delta_x(dx') dx, \end{aligned}$$

and check if the equality in Remark 4.1 holds. For the first term, we can easily see that

$$\begin{aligned} p_{\star}(x) q(x'|x) \alpha(x, x') &= p_{\star}(x) q(x'|x) \min \left\{ 1, \frac{p_{\star}(x') q(x|x')}{p_{\star}(x) q(x'|x)} \right\} \\ &= \min \{ p_{\star}(x) q(x'|x), p_{\star}(x') q(x|x') \} \\ &= \min \left\{ \frac{p_{\star}(x) q(x'|x)}{p_{\star}(x') q(x|x')}, 1 \right\} p_{\star}(x') q(x|x'), \\ &= p_{\star}(x') q(x|x') \alpha(x', x). \end{aligned}$$

Since the second term involves a Dirac measure, we will check it against the integral form directly:

$$\begin{aligned} \int f(x, x') p_{\star}(x) (1 - a(x)) \delta_x(dx') dx &= \int f(x, x) p_{\star}(x) (1 - a(x)) dx \\ &= \int f(x', x') p_{\star}(x') (1 - a(x')) dx' \\ &= \int f(x, x') p_{\star}(x') (1 - a(x')) \delta_{x'}(dx) dx'. \end{aligned}$$

All in all, we can see that

$$\int f(x, x') p_{\star}(x) K(x'|x) dx dx' = \int f(x, x') p_{\star}(x') K(x|x') dx dx',$$

which is the detailed balance condition. □

One can see that the algorithm works just the same with unnormalised densities, i.e., recall

$$p_{\star}(x) = \frac{\bar{p}_{\star}(x)}{Z},$$

where Z is the normalisation constant. In this case, the acceptance ratio becomes

$$r(x, x') = \frac{\bar{p}_{\star}(x') q(x|x')}{\bar{p}_{\star}(x) q(x'|x)},$$

without any change as the normalising constants cancel out in (4.4). We will next describe certain classes of proposals to sample from various kinds of distributions and assess their performance.

4.3.1 INDEPENDENT PROPOSALS

An important class of proposals that is used in practice is the independent proposal. Note that in general we denoted our proposal with $q(x'|x)$, in particular, we would sample from $q(x'|x_{n-1})$ implying that in general the proposal uses the current state of the chain. This does not have to be the case and we can as well chose just an independent proposal $q(x')$ to ease computations. The acceptance ratio in this specific case becomes

$$r(x, x') = \frac{\bar{p}_\star(x')q(x)}{\bar{p}_\star(x)q(x')}.$$

In the algorithm, this means that we compute

$$\alpha(X_{n-1}, X') = \min \left\{ 1, \frac{\bar{p}_\star(X')q(X_{n-1})}{\bar{p}_\star(X_{n-1})q(X')} \right\}.$$

We will see one example as follows.

Example 4.5 (Independent Gaussian proposal). Consider a Gaussian (artificial) target:

$$p_\star(x) = \mathcal{N}(x; \mu, \sigma^2)$$

Assume we want to use MH to sample from it. Choose a proposal

$$q(x) = \mathcal{N}(x; \mu_q, \sigma_q^2).$$

Compute the acceptance ratio.

Solution. The acceptance ratio can be computed in this case as:

$$\begin{aligned} r(x, x') &= \frac{p_\star(x')q(x)}{p_\star(x)q(x')} \\ &= \frac{\mathcal{N}(x'; \mu, \sigma^2)\mathcal{N}(x; \mu_q, \sigma_q^2)}{\mathcal{N}(x; \mu, \sigma^2)\mathcal{N}(x'; \mu_q, \sigma_q^2)} \\ &= \frac{\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x'-\mu)^2}{2\sigma^2}\right) \frac{1}{\sqrt{2\pi\sigma_q^2}} \exp\left(-\frac{(x-\mu_q)^2}{2\sigma_q^2}\right)}{\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \frac{1}{\sqrt{2\pi\sigma_q^2}} \exp\left(-\frac{(x'-\mu_q)^2}{2\sigma_q^2}\right)} \\ &= \frac{\exp\left(-\frac{(x'-\mu)^2}{2\sigma^2}\right) \exp\left(-\frac{(x-\mu_q)^2}{2\sigma_q^2}\right)}{\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \exp\left(-\frac{(x'-\mu_q)^2}{2\sigma_q^2}\right)} \\ &= e\left(-\frac{1}{2\sigma^2}[(x'-\mu)^2 - (x-\mu)^2]\right) e\left(-\frac{1}{2\sigma_q^2}[(x-\mu_q)^2 - (x'-\mu_q)^2]\right) \end{aligned}$$

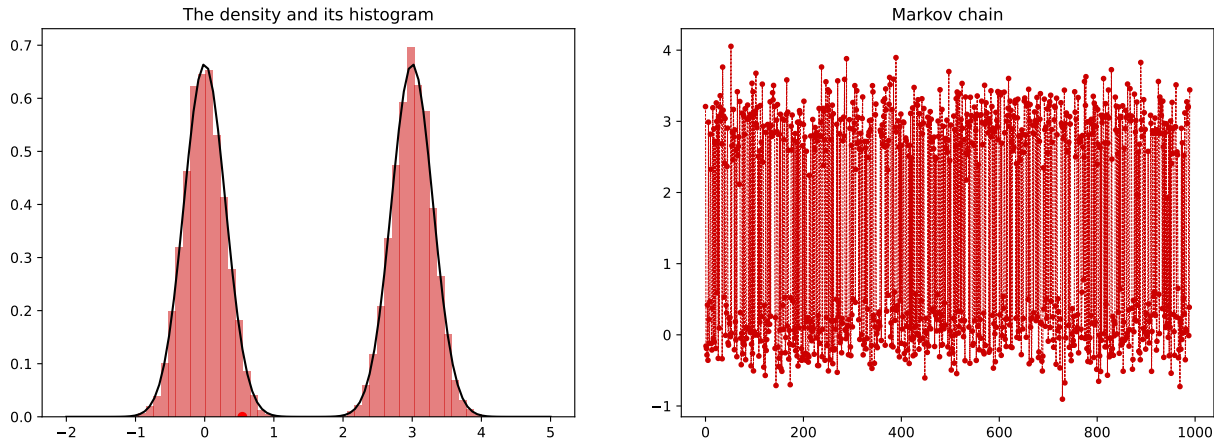


Figure 4.2: Random walk Gaussian proposal for a mixture of two Gaussians.

4.3.2 RANDOM WALK (SYMMETRIC) PROPOSALS

Another important class of proposals is the random walk proposal. In this case, the proposal does use the current state X_{n-1} to define a proposal $q(x'|x_{n-1})$. These proposals in the random walk (and more generally symmetric) case result in a density that is symmetric, i.e., $q(x'|x) = q(x|x')$. This leads to a considerable simplification in the acceptance ratio calculations. We will see some examples below.

Example 4.6 (Random walk Gaussian proposal). Consider a mixture Gaussian target:

$$p_{\star}(x) = w_1 \mathcal{N}(x; \mu_1, \sigma_1^2) + w_2 \mathcal{N}(x; \mu_2, \sigma_2^2),$$

with some $w_1, w_2 > 0$ and $w_1 + w_2 = 1$. Assume we want to use MH to sample from it. Choose a proposal

$$q(x'|x) = \mathcal{N}(x'; x, \sigma_q^2),$$

and derive the acceptance ratio.

Solution. This proposal is symmetric so we can write

$$\begin{aligned} r(x, x') &= \frac{p_{\star}(x')q(x|x')}{p_{\star}(x)q(x'|x)} \\ &= \frac{p_{\star}(x')}{p_{\star}(x)}, \\ &= \frac{w_1 \mathcal{N}(x'; \mu_1, \sigma_1^2) + w_2 \mathcal{N}(x'; \mu_2, \sigma_2^2)}{w_1 \mathcal{N}(x; \mu_1, \sigma_1^2) + w_2 \mathcal{N}(x; \mu_2, \sigma_2^2)}, \end{aligned}$$

which is a considerable simplification. See Fig. 4.2 for a demonstration.

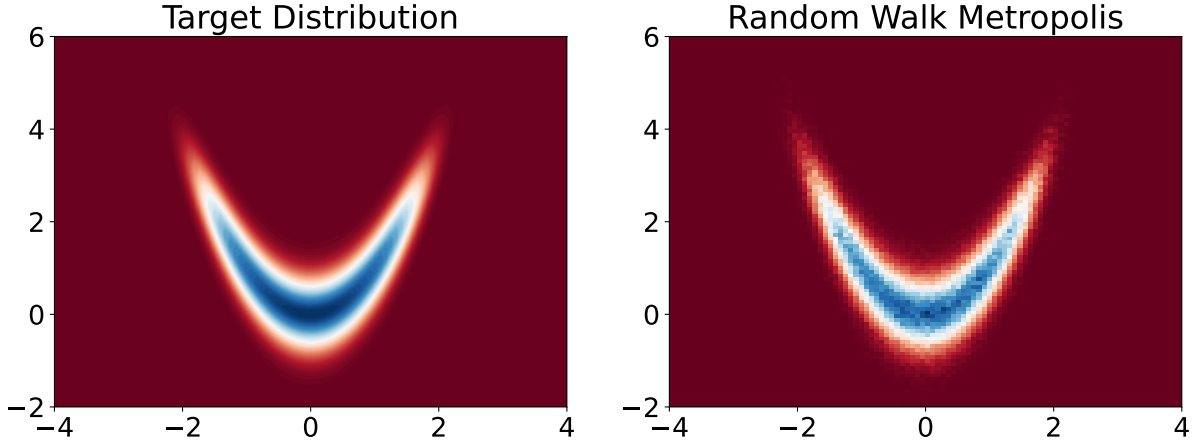


Figure 4.3: Banana density estimation using Random walk metropolis and plotting the histogram.

Example 4.7 (The banana density). Consider the following density:

$$p(x, y) \propto \exp \left(-\frac{x^2}{10} - \frac{y^4}{10} - 2(y - x^2)^2 \right).$$

This is only available in unnormalised form and it is an excellent test problem for many algorithms to fail. Design an MH algorithm for this density.

Solution. We have

$$\bar{p}_*(x, y) = \exp \left(-\frac{x^2}{10} - \frac{y^4}{10} - 2(y - x^2)^2 \right).$$

and let us choose the proposal

$$q(x', y' | x, y) = \mathcal{N}(x'; x, \sigma_q^2) \mathcal{N}(y'; y, \sigma_q^2).$$

This is a symmetric proposal so the acceptance ratio is

$$r(x, y, x', y') = \frac{\bar{p}_*(x', y')}{\bar{p}_*(x, y)}.$$

Note that it makes sense to only compute log-acceptance ratio here

$$\log r(x, y, x', y') = \log \bar{p}_*(x', y') - \log \bar{p}_*(x, y),$$

and implement the acceptance rate by drawing $U \sim \text{Unif}(0, 1)$ and accepting if $\log U < \log r(x, y, x', y')$. The result can be seen from Fig. 4.3.

4.3.3 GRADIENT BASED (LANGEVIN) PROPOSALS

One of the powerful proposal alternatives is to choose the proposal based on the gradient of the target distribution p_* . Note that we can compute $\nabla \log p_*(x)$ without necessarily needing the normalising constant, since

$$\nabla \log p_*(x) = \nabla \log \bar{p}_*(x) - \underbrace{\nabla \log Z}_0$$

Therefore, without doing much more than what we are already doing (using unnormalised density), we can *inform* the proposal by using the gradient of the target distribution:

$$q(x'|x) = \mathcal{N}(x'; x + \gamma \nabla \log p_*(x), 2\gamma I),$$

This algorithm is widely popular in the fields of statistics and especially in machine learning. This approach is called *Metropolis adjusted Langevin algorithm* (MALA)

Exercise 4.4. Sample from the banana density using MH sampler:

$$p(x, y) \propto \exp\left(-\frac{x^2}{10} - \frac{y^4}{10} - 2(y - x^2)^2\right).$$

1. Use a symmetric random walk proposal for each dimension

$$q(x', y'|x, y) = \mathcal{N}(x'; x, \sigma_q^2) \mathcal{N}(y'; y, \sigma_q^2).$$

Compute your acceptance ratio using log density and accept if $\log U < \log r$ (for practice).

2. Use the Metropolis-adjusted Langevin algorithm proposal (MALA).

Use the following snippet for 2D plotting:

```
1 x_bb = np.linspace(-4, 4, 100)
2 y_bb = np.linspace(-2, 6, 100)
3 X_bb, Y_bb = np.meshgrid(x_bb, y_bb)
4 Z_bb = np.exp(banana(X_bb, Y_bb)) # your banana function
5 plt.subplot(1, 3, 1)
6 plt.contourf(X_bb, Y_bb, Z_bb, 100, cmap='RdBu')
7 plt.subplot(1, 3, 2)
8 plt.hist2d(samples_RW[0, burnin:n], samples_RW[1, burnin:n], 100, cmap='
    RdBu', range=[[-4, 4], [-2, 6]],
    density=True)
9 plt.title('Random Walk Metropolis')
10 plt.subplot(1, 3, 3)
11 plt.hist2d(samples_Langevin[0, burnin:n], samples_Langevin[1, burnin:n],
    100, cmap='RdBu', range=[[-4, 4], [-2
    , 6]], density=True)
12 plt.title('Metropolis Adjusted Langevin Algorithm')
13 plt.show()
```

4.3.4 BAYESIAN INFERENCE WITH METROPOLIS-HASTINGS

We can finally use the Metropolis-Hastings method for Bayesian inference. In what follows, we will provide some examples for this and visualisations resulting from the sampling procedures.

Recall that, with conditionally independent observations $y_1, \dots, y_n$, we have the Bayes theorem as

$$p(x|y_{1:n}) = \frac{p(y_{1:n}|x)p(x)}{p(y_{1:n})} = \frac{\prod_{i=1}^n p(y_i|x)p(x)}{p(y_{1:n})}.$$

We write

$$p(x|y_{1:n}) \propto \prod_{i=1}^n p(y_i|x)p(x),$$

and set

$$\bar{p}_\star(x) = \prod_{i=1}^n p(y_i|x)p(x),$$

which is the unnormalised posterior density. We can then use the Metropolis-Hastings algorithm to sample from this posterior density. A generic Metropolis-Hastings method for Bayesian inference is described in Algorithm 10.

Algorithm 10 Pseudocode for Metropolis Hastings method for Bayesian inference

- 1: Input: The number of samples N , and starting point X_0 .
- 2: **for** $i = 1, \dots, N$ **do**
- 3: Propose (sample): $X' \sim q(x'|X_{n-1})$
- 4: Accept the sample $X_n = X'$ with probability

$$\alpha(X_{n-1}, X') = \min \left\{ 1, \frac{\bar{p}_\star(x')q(x_{n-1}|x')}{\bar{p}_\star(x_{n-1})q(x'|x_{n-1})} \right\}.$$

- 5: Otherwise reject the sample and set $X_n = X_{n-1}$.
 - 6: **end for**
 - 7: Discard first burnin samples and return the remaining samples.
-

Example 4.8 (Source localisation). This example is taken from [Cemgil \(2014\)](#). Consider the problem of source localisation in the presence of three sensors with three noisy observations. The setup in this example can be seen from the left part of Fig. 4.4. We have three sensors surrounding an object we are trying to locate. The sensors receive noisy observations on $\mathbb{R}^2$. We are trying to locate the object based on these observations. We define our prior rather broadly: $p(x) = \mathcal{N}(x; \mu, \sigma^2 I)$ where $\mu = [0, 0]$ and $\sigma^2 = 20$. We assume that the observations are coming from

$$p(y_i|x, s_i) = \mathcal{N}(y_i; \|x - s_i\|, \sigma_y^2),$$

where s_i is the location of the i th sensor on $\mathbb{R}^2$ for $i = 1, 2, 3$. We assume that the observations are independent and that the noise is independent of the location of the object (of course, for the sake of the example, we simulate our observations from the true model which is not the case in the real world). Devise a MH sampler to sample from the posterior density of x , i.e., the distribution over the location of the hidden object.

Solution. We can write the posterior density as

$$p(x|y_1, y_2, y_3, s_1, s_2, s_3) \propto p(y_1, y_2, y_3|x, s_1, s_2, s_3)p(x),$$

and given conditional independence we have

$$p(x|y_1, y_2, y_3, s_1, s_2, s_3) \propto p(x) \prod_{i=1}^3 p(y_i|x, s_i).$$

This sort of Bayes update follows from the conditional Bayes rule introduced in Prop. 1.1. In order to design the MH scheme, therefore, we need to just evaluate the likelihood and the prior for MH. We choose a random walk proposal:

$$q(x'|x) = \mathcal{N}(x'; x, \sigma^2 I).$$

This is symmetric so the acceptance ratio is:

$$r(x, x') = \frac{p(x')p(y_1|x', s_1)p(y_2|x', s_2)p(y_3|x', s_3)}{p(x)p(y_1|x, s_1)p(y_2|x, s_2)p(y_3|x, s_3)}.$$

An example solution to this problem can be seen from Fig. 4.4 and the code can be accessed from our online companion.

Example 4.9 (Gaussian with unknown mean and variance Example 5.13 in Yıldırım (2017)). Assume that we observe

$$Y_1, \dots, Y_n | z, s \sim \mathcal{N}(y_i; z, s)$$

where we do not know z and s . Assume we have an independent prior on z and s :

$$p(z)p(s) = \mathcal{N}(z; m, \kappa^2) \mathcal{IG}(s; \alpha, \beta).$$

where $\mathcal{IG}(s; \alpha, \beta)$ is the inverse Gamma distribution

$$\mathcal{IG}(s; \alpha, \beta) = \frac{\beta^\alpha}{\Gamma(\alpha)} s^{-\alpha-1} \exp\left(-\frac{\beta}{s}\right).$$

Design an MH algorithm to sample from the posterior distribution $p(z, s|y_{1:n})$.

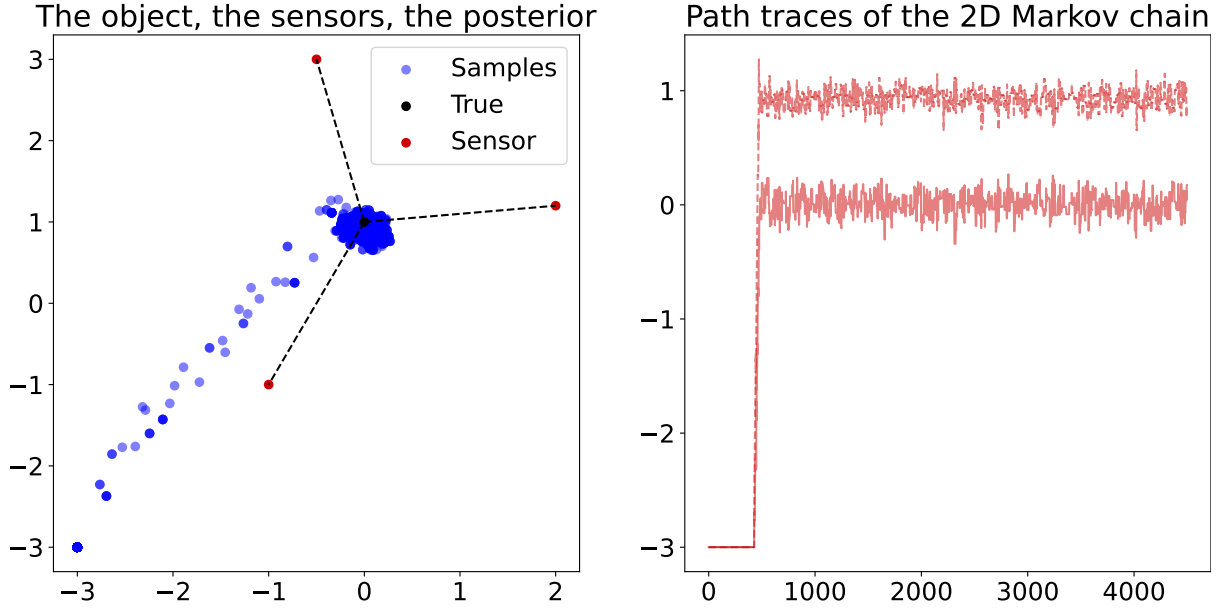


Figure 4.4: Solution of the source localisation problem.

Solution. We can explicitly write the priors as

$$p(z)p(s) = \frac{1}{\sqrt{2\pi\kappa^2}} \exp\left(-\frac{(z-m)^2}{2\kappa^2}\right) \frac{\beta^\alpha}{\Gamma(\alpha)} s^{-\alpha-1} \exp\left(-\frac{\beta}{s}\right).$$

We are after the posterior distribution

$$\begin{aligned} p(z, s | y_1, \dots, y_n) &\propto p(y_1, \dots, y_n | z, s) p(z) p(s), \\ &= \prod_{i=1}^n \mathcal{N}(y_i; z, s) \mathcal{N}(z; m, \kappa^2) \mathcal{IG}(s; \alpha, \beta). \end{aligned}$$

Let us call our unnormalised posterior as $\bar{p}_*(z, s | y_{1:n})$. In order to do this, we need to design proposals over z and s . We choose a random walk proposal for z :

$$q(z' | z) = \mathcal{N}(z'; z, \sigma_q^2).$$

and an independent proposal for s :

$$q(s') = \mathcal{IG}(s'; \alpha, \beta).$$

The joint proposal therefore is

$$q(z', s' | z, s) = \mathcal{N}(z'; z, \sigma_q^2) \mathcal{IG}(s'; \alpha, \beta).$$

When we design the MH algorithm, we see that the acceptance ratio is

$$\begin{aligned} r(z, s, z', s') &= \frac{\bar{p}_*(z', s' | y_{1:n}) q(z, s | z', s')}{\bar{p}_*(z, s | y_{1:n}) q(z', s' | z, s)} \\ &= \frac{p(z') p(s') [\prod_{k=1}^n \mathcal{N}(y_k; z', s')]}{p(z) p(s) [\prod_{k=1}^n \mathcal{N}(y_k; z, s)]} \frac{\mathcal{N}(z; z', \sigma_q^2) p(s)}{\mathcal{N}(z'; z, \sigma_q^2) p(s')} \\ &= \frac{\mathcal{N}(z'; m, \kappa^2) [\prod_{k=1}^n \mathcal{N}(y_k; z', s')]}{\mathcal{N}(z; m, \kappa^2) [\prod_{k=1}^n \mathcal{N}(y_k; z, s)]} \end{aligned}$$

Exercise 4.5. Consider a generic target $p(x)$ and a proposal $q(x)$. Consider two algorithms: (i) The rejection sampler with proposal $q(x)$ and (ii) the independent MH sampler with proposal $q(x)$. Assume that M is chosen optimally for the rejection sampler. Prove that, for a given x , the MH acceptance ratio is always higher than the rejection sampler acceptance ratio, i.e., the MH algorithm accepts more often than the rejection sampler.

4.4 GIBBS SAMPLING

We will now go into another major class of MCMC samplers, called Gibbs samplers. The idea of Gibbs samplers is that, given a joint distribution of many variables $p(x_1, \dots, x_d)$, one can build a Markov chain that samples from this distribution by sampling from the conditional distributions. This will also allow us straightforwardly sample from high-dimensional distributions. The downside of this approach is that, one has to derive the conditional distributions, which can be difficult. However, if one can do this, then the Gibbs sampler can be a very efficient method.

In this chapter, we denote our target similarly as $p_\star(x)$ where $x \in \mathbb{R}^d$ and define the *full conditional* distributions as

$$p_{m,\star}(x_m | x_1, \dots, x_{m-1}, x_{m+1}, \dots, x_d) = p_{m,\star}(x_m | x_{1:m-1}, x_{m+1:d}) = p_{m,\star}(x_m | x_{-m}),$$

where $x_{-m} = (x_1, \dots, x_{m-1}, x_{m+1}, \dots, x_d)$ is the vector of all variables except x_m . For a moment, assume that the full conditionals are available. Also assume, we obtain $X_{n-1} \in \mathbb{R}^d$ at the $n-1$ 'th iteration of the algorithm. To denote individual components, we use $X_{n-1,m}$ to denote the m 'th component of X_{n-1} . Of course, the key aspect of the Gibbs sampler is to derive the

Algorithm 11 Pseudocode for the Gibbs sampler

- 1: Input: The number of samples N , and starting point $X_0 \in \mathbb{R}^d$.
- 2: **for** $n = 1, \dots, N$ **do**
- 3: Sample

$$\begin{aligned} X_{n,1} &\sim p_{1,\star}(X_{n,1} | X_{n-1,2}, \dots, X_{n-1,d}) \\ X_{n,2} &\sim p_{2,\star}(X_{n,2} | X_{n,1}, X_{n-1,3}, \dots, X_{n-1,d}) \\ X_{n,3} &\sim p_{3,\star}(X_{n,3} | X_{n,1}, X_{n,2}, X_{n-1,4}, \dots, X_{n-1,d}) \\ X_{n,d} &\sim p_{d,\star}(X_{n,d} | X_{n,1}, X_{n,2}, \dots, X_{n,d-1}) \end{aligned}$$

- 4: **end for**
 - 5: Discard first burnin samples and return the remaining samples.
-

full conditional distributions. We will come back to this point, but we will first investigate why the Gibbs sampling approach provides us a valid MCMC kernel, in other words, how the Gibbs sampler satisfies the detailed balance.

Let us denote $x = (x_m, x_{-m})$. It is easy to see from Algorithm 11 that the Gibbs sampler for every iteration (at time n) is defined as d separate operations, each sampling from the conditional

distribution. We can first look at what goes on in each of these d updates. It is also easy to see that, the kernel defined in each of these d updates is given as

$$K_m(x'|x) = p_{m,\star}(x'_m|x_{-m})\delta_{x_{-m}}(x'_{-m}),$$

where $\delta_{x_{-m}}(x'_{-m})$ is the Dirac delta function. Intuitively, each step samples from the full conditional $p_{m,\star}(\cdot|x_{-m})$ for m th dimension where $m \in \{1, \dots, d\}$ and leaves others unchanged, which is enforced by the term $\delta_{x_{-m}}(x'_{-m})$. One can then see that the entire Gibbs kernel can be written as

$$K = K_1 K_2 \dots K_d.$$

Note that each kernel is an *integral operator* – therefore the above equation is almost symbolic, it does not mean multiplication of kernels. We will now show that the Gibbs kernel satisfies the detailed balance.

Proposition 4.3. *The Gibbs kernel K leaves the target distribution $p_\star$ invariant.*

Proof. We first show that each kernel K_m satisfies the detailed balance condition:

$$\begin{aligned} p_\star(x)K_m(x'|x) &= p_\star(x)p_{m,\star}(x'_m|x_{-m})\delta_{x_{-m}}(x'_{-m}) \\ &= p_\star(x_{-m})p_{m,\star}(x_m|x_{-m})p_{m,\star}(x'_m|x_{-m})\delta_{x_{-m}}(x'_{-m}) \\ &= p_\star(x'_{-m})p_{m,\star}(x'_m|x'_{-m})p_{m,\star}(x_m|x'_{-m})\delta_{x'_{-m}}(x_{-m}) \\ &= p_\star(x')K_m(x|x'). \end{aligned}$$

The steps of this derivation follows from the fact that the use of Dirac allows us to exchange variables x and x' . This shows that K_m satisfies the detailed balance condition, therefore, we have

$$\int K_m(x'|x)p_\star(x)dx = p_\star(x'),$$

i.e., K_m leaves $p_\star$ invariant. Let us denote now the integral

$$(K_m, p_\star) = \int K_m(x'|x)p_\star(x)dx = p_\star.$$

One can see that we have $(K_2, (K_1, p_\star)) = (K_2, p_\star) = p_\star$, which is true for all $m = 1, \dots, d$. Therefore, we see that application of d kernels $K_1, \dots, K_d$ will leave $p_\star$ invariant. $\square$

We can also see why Gibbs sampling works by relating it to the Metropolis-Hastings algorithm. Recall that we can see our sampling from the conditional as a proposal, i.e.,

$$q_m(x'|x) = p_{m,\star}(x'_m|x_{-m})\delta_{x_{-m}}(x'_{-m}).$$

If we calculate the acceptance ratio for this proposal

$$\begin{aligned} \alpha_m(x'|x) &= \min \left\{ 1, \frac{p_\star(x')q_m(x|x')}{p_\star(x)q_m(x'|x)} \right\}, \\ &= \min \left\{ 1, \frac{p_\star(x')K_m(x|x')}{p_\star(x)K_m(x'|x)} \right\}. \end{aligned}$$

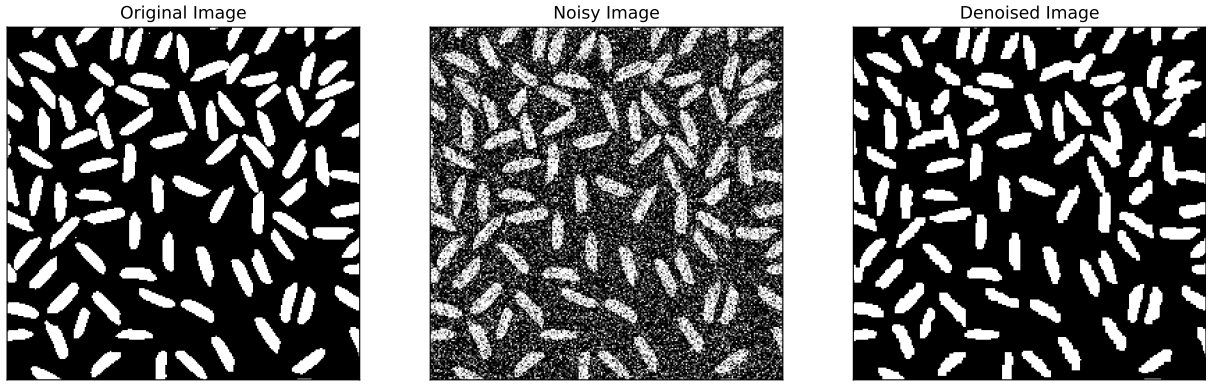


Figure 4.5: Denoising of an image using Gibbs sampler. The left column shows the original image, the middle column shows the noisy image, and the right column shows the denoised image. I used $\sigma = 1$, $J = 4$ for this and the Gibbs sampler scanned the entire image only 10 times.

We see that this is equal to 1 as the detailed balance is satisfied for q_m (which is K_m – see the proof of Proposition 4.3).

As we noted before, we have shown that the kernel K would leave p_* invariant, but this would not give us proper convergence guarantees. Note that the version of the algorithm we presented is called *deterministic scan* Gibbs sampler. The reason for this is that the algorithm in Alg. 11 is implemented so that we sample $x_1, \dots, x_d$ in order, *scanning* the variables deterministically. It turns out, while this sampler’s convergence guarantees cannot be established easily, there is an algorithmic fix which results in a procedure that is also guaranteed to converge. Instead of scanning the variables deterministically, we can sample them in a random order. This is called the *random scan* Gibbs sampler. The algorithm is given in Alg. 12. We will now see an example.

Algorithm 12 Random scan Gibbs sampler

- 1: Input: The number of samples N , and starting point $X_0 \in \mathbb{R}^d$.
- 2: **for** $n = 1, \dots, N$ **do**
- 3: Sample $j \sim \{1, \dots, d\}$

$$X_{n,j} \sim p_{j,\star}(X_{n,j} | X_{n,-j}),$$

- 4: **end for**
-

Example 4.10 (Image denoising). A biologist knocked your door as some of the images from the microscope were too noisy. You decide to help and use the Gibbs sampler for this. The model is given as follows.

Consider a set of random variables X_{ij} for $i = 1, \dots, m$ and $j = 1, \dots, n$. This is a matrix modeling an $m \times n$ image. We assume that we have an image that takes values $X_{ij} \in \{-1, 1\}$ – note that this is an “unusual” image, as the images usually take values between $[0, 255]$ (or $[0, 1]$). We assume that the image is corrupted by noise, i.e., we have a noisy image

$$Y_{ij} = X_{ij} + \sigma \epsilon_{ij},$$

where $\epsilon_{ij} \sim \mathcal{N}(0, 1)$ and σ is the standard deviation of the noise. We assume that the noise is independent of the image. We want to recover the image X_{ij} from the noisy image Y_{ij} and utilise Gibbs sampler for this purpose.

The goal is obtain (conceptually) $p(X|Y)$, i.e., samples from $p(X|Y)$ given Y . For this, we need to specify a prior $p(X)$. We take this from the literature and place as a prior a smooth Markov random field (MRF) assumption. This is formalised as

$$p(X_{ij}|X_{-ij}) = \frac{1}{Z} \exp(JX_{ij}W_{ij}),$$

where W_{ij} is the sum of the X_{ij} 's in the neighbourhood of X_{ij} , i.e.,

$$W_{ij} = \sum_{kl: \text{neighbourhood of } (i,j)} X_{kl} = X_{i-1,j} + X_{i+1,j} + X_{i,j-1} + X_{i,j+1}.$$

This is an intuitive model of the image, making the current value of the pixel depend on the values of its neighbours. The exercise is to design the Gibbs sampler for this problem.

Solution. We aim at using a Gibbs sampler approach from sampling the posterior $p(X|Y)$. Note that now we need to sample from full conditionals, e.g., for each (i, j) , we need to sample from $X_{ij} \sim p(X_{ij}|X_{-ij}, Y_{ij})$. We derive the full conditional as

$$p(X_{ij} = k|X_{-ij}, Y_{ij}) = \frac{p(Y_{ij}|X_{ij} = k)p(X_{ij} = k|X_{-ij})}{\sum_{k \in \{-1, 1\}} p(Y_{ij}|X_{ij} = k)p(X_{ij} = k|X_{-ij})},$$

where $p(Y_{ij}|X_{ij} = k) = \mathcal{N}(Y_{ij}; k, \sigma^2)$ is the likelihood of the noisy image given the value of the pixel. We can easily compute these probabilities since each term in the Bayes rule is computable (and $1/Z$ cancels). Therefore, we can get explicit expressions for $q = p(X_{ij} = 1|X_{-ij}, Y_{ij})$ and $1 - q = p(X_{ij} = -1|X_{-ij}, Y_{ij})$. We can then sample from the full conditional as

$$X_{ij} \sim \begin{cases} 1 & \text{with probability } q, \\ -1 & \text{with probability } 1 - q. \end{cases}$$

We can now loop over (i, j) (to sample from each full conditional) and sample from the full conditionals. This is the Gibbs sampler algorithm as described above. The results of this procedure can be seen from Fig. 4.5.

Let us consider another example.

Example 4.11 (Beta-Binomial Gibbs sampler). Consider the following model

$$p(\theta) = \text{Beta}(\theta; \alpha, \beta) = \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} \theta^{\alpha-1} (1 - \theta)^{\beta-1},$$

and

$$p(x|\theta) = \text{Bin}(x; n, \theta) = \binom{n}{x} \theta^x (1 - \theta)^{n-x}.$$

Derive the Gibbs sampler to sample from the joint distribution $p(x, \theta)$.

Solution. We know that, for this, we need full conditionals, i.e., we need $p(x|\theta)$ and $p(\theta|x)$. We can see that $p(x|\theta)$ is already provided in the definition of the model. Therefore, we only need to derive the posterior. We can write the joint distribution as

$$\begin{aligned} p(x, \theta) &= p(x|\theta)p(\theta) = \binom{n}{x} \theta^x (1 - \theta)^{n-x} \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} \theta^{\alpha-1} (1 - \theta)^{\beta-1}, \\ &= \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} \binom{n}{x} \theta^{x+\alpha-1} (1 - \theta)^{n-x+\beta-1}. \end{aligned}$$

For Bayes theorem $p(\theta|x) = p(x|\theta)p(\theta)/p(x)$, we also need to compute $p(x)$. This is given by

$$\begin{aligned} p(x) &= \int_0^1 p(x, \theta) d\theta = \int_0^1 \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} \binom{n}{x} \theta^{x+\alpha-1} (1 - \theta)^{n-x+\beta-1} d\theta, \\ &= \binom{n}{x} \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} \frac{\Gamma(x + \alpha)\Gamma(n - x + \beta)}{\Gamma(n + \alpha + \beta)}. \end{aligned}$$

Therefore, we can compute the posterior as

$$\begin{aligned} p(\theta|x) &= \frac{p(x, \theta)}{p(x)} = \frac{\binom{n}{x} \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} \theta^{x+\alpha-1} (1 - \theta)^{n-x+\beta-1}}{\binom{n}{x} \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} \frac{\Gamma(x + \alpha)\Gamma(n - x + \beta)}{\Gamma(n + \alpha + \beta)}}, \\ &= \text{Beta}(\theta; x + \alpha, n - x + \beta). \end{aligned}$$

Therefore we can sample from $p(\theta|x)$ using any method to simulate a Beta variable. The Gibbs sampler is then defined as follows:

- Initialise x_0, θ_0
- For $k = 1, 2, \dots$:
 - Sample $\theta_k \sim p(\theta|x_{k-1})$
 - Sample $x_k \sim p(x|\theta_k)$
- Return x_k, θ_k for $k = 1, 2, \dots$

We also note that simulated x_k are approximately from $p(x)$ which also gives us a way to approximate $p(x)$.

Exercise 4.6. Assume that you are given conditional distributions

$$\begin{aligned} p(x|y) &= \text{Exp}(y) \\ p(y|x) &= \text{Exp}(x), \end{aligned}$$

and you have implemented the Gibbs sampler using these conditionals.

1. Write the Gibbs sampler algorithm (on paper) assuming only accessing uniform random numbers.
2. Now show that these conditionals are consistent with a target of the form $p(x, y) \propto \exp(-xy)$.
3. Show that the “target density” is not a valid density. What issue does this highlight about Gibbs sampling implementations?
4. Finally, assume that we have the joint density $p(x, y)$ in the same form as you derived in Part 2, but constrained to $(x, y) \in (0, 1) \times (0, 1)$. In this case, derive the Gibbs sampler by deriving full conditionals.

4.4.1 METROPOLIS-WITHIN-GIBBS

One remarkable feature of the Gibbs sampler is that when we cannot derive the full conditionals (or too lazy to do it), we can instead target the full conditional with a single Metropolis step at each iteration. This is called the Metropolis-within-Gibbs algorithm and, remarkably, it samples from the correct posterior!

Consider a generic target $p(x, y)$. In many situations, it is easier to write *unnormalised* full conditionals, i.e., we would have access to $\bar{p}(x|y)$ and $\bar{p}(y|x)$, but not $p(x|y)$ and $p(y|x)$. In this case, we can use the Metropolis-within-Gibbs algorithm – meaning that instead of sampling from the full conditional, we can take a Metropolis step to sample from the full conditional. The algorithm would be summarised as follows. Given (x_{n-1}, y_{n-1})

1. Sample $x' \sim q_x(x'|x_{n-1})$ and accept x' with probability

$$r_x = \frac{\bar{p}(x'|y_{n-1})q_x(x_{n-1}|x')}{\bar{p}(x_{n-1}|y_{n-1})q_x(x'|x_{n-1})}$$

i.e., set $x_n = x'$ with probability r_x and $x_n = x_{n-1}$ otherwise.

2. Sample $y' \sim q_y(y'|y_{n-1})$ and accept y' with probability

$$r_y = \frac{\bar{p}(y'|x_n)q_y(y_{n-1}|y')}{\bar{p}(y_{n-1}|x_n)q_y(y'|y_{n-1})}$$

i.e., set $y_n = y'$ with probability r_y and $y_n = y_{n-1}$ otherwise.

Example 4.12 (Metropolis-within-Gibbs for Example 4.9). Let us return to Example 4.9. To recall the model, assume that we observe

$$Y_1, \dots, Y_n | z, s \sim \mathcal{N}(y_i; z, s)$$

where we do not know z and s . Assume we have an independent prior on z and s :

$$p(z)p(s) = \mathcal{N}(z; m, \kappa^2) \mathcal{IG}(s; \alpha, \beta).$$

where $\mathcal{IG}(s; \alpha, \beta)$ is the inverse Gamma distribution

$$\mathcal{IG}(s; \alpha, \beta) = \frac{\beta^\alpha}{\Gamma(\alpha)} s^{-\alpha-1} \exp\left(-\frac{\beta}{s}\right).$$

In other words, we have

$$p(z)p(s) = \frac{1}{\sqrt{2\pi\kappa^2}} \exp\left(-\frac{(z-m)^2}{2\kappa^2}\right) \frac{\beta^\alpha}{\Gamma(\alpha)} s^{-\alpha-1} \exp\left(-\frac{\beta}{s}\right).$$

We are after the posterior distribution

$$\begin{aligned} p(z, s | y_1, \dots, y_n) &\propto p(y_1, \dots, y_n | z, s) p(z) p(s), \\ &= \prod_{i=1}^n \mathcal{N}(y_i; z, s) \mathcal{N}(z; m, \kappa^2) \mathcal{IG}(s; \alpha, \beta). \end{aligned}$$

Let us call our unnormalised posterior as $\bar{p}_\star(z, s | y_{1:n})$. Now instead of MH or defining Gibbs (requires us to derive full conditionals), derive the Metropolis-within-Gibbs algorithm.

Solution. For this, note the unnormalised full conditionals:

$$\bar{p}_\star(z | s, y_{1:n}) = \prod_{i=1}^n \mathcal{N}(y_i; z, s) \mathcal{N}(z; m, \kappa^2),$$

and

$$\bar{p}_\star(s | z, y_{1:n}) = \prod_{i=1}^n \mathcal{N}(y_i; z, s) \mathcal{IG}(s; \alpha, \beta).$$

In order to do this, we need to design proposals over z and s to target $\bar{p}(z | s, y_{1:n})$ and $\bar{p}(s | z, y_{1:n})$ respectively. This step will be a standard Metropolis as if we are solving each problem independently. We choose a random walk proposal for z :

$$q(z' | z) = \mathcal{N}(z'; z, \sigma_q^2).$$

and an independent proposal for s :

$$q(s') = \mathcal{IG}(s'; \alpha, \beta).$$

Therefore, we Metropolis-within-Gibbs can be implemented as follows

- Initialise z_0, s_0
- For $k = 1, 2, \dots$:
- Metropolis step for z -marginal:
 - Sample $z' \sim q(z'|z_{k-1})$
 - Accept z' and set $z_k = z'$ with probability

$$r_z = \frac{\bar{p}_\star(z'|s_{k-1}, y_{1:n})}{\bar{p}_\star(z_{k-1}|s_{k-1}, y_{1:n})}$$

which is simplified due to the symmetric proposal.

- Otherwise set $z_k = z_{k-1}$.
- Metropolis step for s -marginal:
 - Sample $s' \sim q(s')$
 - Accept s' and set $s_k = s'$ with probability

$$\begin{aligned} r_s &= \frac{\bar{p}_\star(s'|z_k, y_{1:n})q(s_{k-1})}{\bar{p}_\star(s_{k-1}|z_k, y_{1:n})q(s')} \\ &= \frac{\prod_{i=1}^n \mathcal{N}(y_i; z, s')}{\prod_{i=1}^n \mathcal{N}(y_i; z, s_{k-1})} \end{aligned}$$

- Otherwise set $s_k = s_{k-1}$.
- Return z_k, s_k for $k = 1, 2, \dots$

4.5 LANGEVIN MCMC METHODS

We briefly introduced Metropolis-adjusted Langevin algorithm (MALA) in Sec. 4.3.3. MALA is just one example of a more general class of samplers, called Langevin MCMC algorithms, which are based on Langevin dynamics. These approaches are at the forefront of modern MCMC methods, used in a variety of settings, including sampling from high-dimensional targets, deep learning, sampling from Bayesian neural networks, and so on. We will now introduce the Langevin MCMC methods and see how they work.

Consider again our target $p_\star$ defined on $\mathbb{R}^d$. It turns out, we can use stochastic differential equations (SDEs) to sample from $p_\star$ (recall that SDEs are differential equations with a stochastic term). Consider the following SDE:

$$dX_t = \nabla \log p_\star(X_t)dt + \sqrt{2}dB_t, \quad (4.5)$$

where B_t is a standard Brownian motion. It turns out the marginal distributions of X_t driven by this SDE converge to $p_\star$ as $t \rightarrow \infty$. In other words, in many suitable metrics, we can quantify the convergence $d(p_t, p_\star) \rightarrow 0$ as $t \rightarrow \infty$. This means that all we need to draw samples from $p_\star$ is to

numerically solve this SDE which can be done with a variety of numerical methods (akin to ODE solvers). One caveat in this situation is that, while the SDE would target p_* , its discretisation would incur bias. This is why MALA is “Metropolised”.

Let us recall the MALA algorithm. We start with a point X_0 and then define the proposal

$$q(x_n|x_{n-1}) = \mathcal{N}(x_n; x_{n-1} + \gamma \nabla \log p_*(x_{n-1}), 2\gamma I_d), \quad (4.6)$$

where $\gamma > 0$ is a step size. We then sample from q to obtain X_n . We then accept X_n with probability

$$\alpha(X_n, X_{n-1}) = \min \left(1, \frac{p_*(X_n)q(X_{n-1}|X_n)}{p_*(X_{n-1})q(X_n|X_{n-1})} \right). \quad (4.7)$$

Recall that the MALA proposal would not define a symmetric proposal, therefore, we would need to compute the ratio. Now one can see that, Eq. (4.6) can be equivalently written as

$$X_n = X_{n-1} + \gamma \nabla \log p_*(X_{n-1}) + \sqrt{2\gamma} W_n, \quad (4.8)$$

where $W_n \sim \mathcal{N}(0, I)$. The relationship of Eq. (4.8) to (4.5) can be seen by noting that the discretisation of the SDE in (4.5) would exactly take the form of Eq. (4.8). Therefore, MALA uses this Langevin SDE as the proposal and then accept/reject its samples. This has a beneficial effect of correcting the bias of the discretisation.

However, the Metropolis step can be computationally infeasible in higher dimensions, just as in the case of rejection sampling. Higher dimensional problems cause the acceptance rate to vanish, which results in slow convergence. To remedy this situation, a common approach is to simply drop the Metropolis step and use the following iteration:

$$X_n = X_{n-1} + \gamma \nabla \log p_*(X_{n-1}) + \sqrt{2\gamma} W_n. \quad (4.9)$$

This is a simple (and biased) MCMC method - which is called the unadjusted Langevin algorithm (ULA). The ULA is a discretisation of the SDE, and as such, its stationary measure is not p_* . However, under various conditions, it can be shown that the limiting distribution of ULA p_*^γ can be made arbitrarily close to p_* as $\gamma \rightarrow 0$. This means that the ULA can be a viable alternative.

Example 4.13. Consider the target $p_*(x) = \mathcal{N}(x; \mu, \sigma^2)$. Write down the ULA for this target and derive the stationary distribution of the chain.

Solution. We can write the ULA as

$$\frac{\partial}{\partial x} \log p_*(x) = -\frac{x - \mu}{\sigma^2}.$$

We can then write the iterates of ULA as

$$\begin{aligned} X_n &= X_{n-1} + \gamma \frac{\partial}{\partial x} \log p_*(X_{n-1}) + \sqrt{2\gamma} W_n, \\ &= X_{n-1} - \gamma \frac{X_{n-1} - \mu}{\sigma^2} + \sqrt{2\gamma} W_n, \\ &= \left(1 - \frac{\gamma}{\sigma^2}\right) X_{n-1} + \frac{\gamma}{\sigma^2} \mu + \sqrt{2\gamma} W_n, \end{aligned}$$

where $W_n \sim \mathcal{N}(0, 1)$. In this simple case, we can compute the stationary distribution of the chain and analyse its relationship to the true target p_* . Let

$$a = 1 - \frac{\gamma}{\sigma^2}, \quad b = \frac{\gamma}{\sigma^2} \mu.$$

We can write now the iterates beginning at x_0 as

$$\begin{aligned} x_1 &= ax_0 + b + \sqrt{2\gamma}W_1, \\ x_2 &= \underbrace{a^2x_0 + ab + a\sqrt{2\gamma}W_1}_{ax_1} + b + \sqrt{2\gamma}W_2, \\ x_3 &= \underbrace{a^3x_0 + a^2b + a^2\sqrt{2\gamma}W_1 + ab + a\sqrt{2\gamma}W_2}_{ax_2} + b + \sqrt{2\gamma}W_3, \\ &\vdots \\ x_n &= a^n x_0 + \sum_{k=0}^{n-1} a^k b + \sum_{k=0}^{n-1} a^k \sqrt{2\gamma} W_{n-k}. \end{aligned}$$

We can compute the expected value

$$\mathbb{E}[X_n] = a^n x_0 + \sum_{k=0}^{n-1} a^k b,$$

since W_k are zero mean. As $n \rightarrow \infty$, we have

$$\mu_\infty = \lim_{n \rightarrow \infty} \mathbb{E}[X_n] = \sum_{k=0}^{\infty} a^k b = \frac{b}{1-a} = \mu.$$

since $0 < a < 1$. The variance of the iterates as $n \rightarrow \infty$ can also be computed. Note that for finite n , we have

$$\begin{aligned} \text{var}(x_n) &= \text{var} \left(\sum_{k=0}^{n-1} a^k \sqrt{2\gamma} W_k \right), \\ &= 2\gamma \sum_{k=0}^{n-1} (a^2)^k, \\ &= 2\gamma \frac{1 - a^{2n}}{1 - a^2}. \end{aligned}$$

Therefore, we obtain the limiting variance as

$$\begin{aligned} \lim_{n \rightarrow \infty} \text{var}(x_n) &= 2\gamma \frac{1}{1 - a^2} \\ &= 2\gamma \frac{1}{1 - \left(1 - \frac{\gamma}{\sigma^2}\right)^2} \\ &= 2\gamma \frac{1}{\frac{2\gamma}{\sigma^2} - \frac{\gamma^2}{\sigma^4}}, \\ &= \frac{2\sigma^4}{2\sigma^2 - \gamma}. \end{aligned}$$

Therefore, we obtained the target measure of ULA as

$$p_{\star}^{\gamma}(x) = \mathcal{N}\left(x; \mu, \frac{2\sigma^4}{2\sigma^2 - \gamma}\right),$$

which is different than $p_{\star}$. Note that in this particular case, the means of the $p_{\star}^{\gamma}$ and $p_{\star}$ agree. It can be seen that the bias enters the picture through the variance, but this vanishes as $\gamma \rightarrow 0$.

Exercise 4.7 (Stochastic Simulation, Final Exam Q3(c), 2025). Consider the following modified unadjusted Langevin algorithm (ULA):

$$X_{k+1} = X_k + \gamma \nabla \log p_{\star}(X_k) + \sigma_q W_k,$$

where $W_k \sim \mathcal{N}(0, 1)$ (assuming the chain is 1D), $\gamma > 0$, and $\sigma_q > 0$. Consider the target distribution:

$$p_{\star}(x) = \mathcal{N}(x; \mu, \sigma^2).$$

- (i) Derive the limiting distribution of the chain $(X_k)_{k \geq 0}$. Provide any necessary constraints on γ for this to be well-defined.
- (ii) Can you make ULA unbiased in the limit with an appropriate choice of σ_q ? If no, provide your reasoning. If yes, provide the expression of σ_q .

Let us look at the ULA for the Banana density.

Example 4.14. Consider the Banana density

$$p(x, y) \propto \exp\left(-\frac{x^2}{10} - \frac{y^4}{10} - 2(y - x^2)^2\right).$$

Derive ULA for this density.

Solution. Note the density is only available in unnormalised form:

$$\bar{p}_{\star}(x, y) = \exp\left(-\frac{x^2}{10} - \frac{y^4}{10} - 2(y - x^2)^2\right).$$

Recall that $\nabla \log \bar{p}_{\star}(x, y) = \nabla \log p_{\star}(x, y)$. Therefore, we will directly compute the unnormalised gradients:

$$\nabla \log p_{\star}(x, y) = \begin{bmatrix} \frac{-x}{5} + 8x(y - x^2) \\ \frac{-2y^3}{5} - 4(y - x^2) \end{bmatrix}.$$

Therefore, the update in 2D is given by

$$\begin{bmatrix} x_{n+1} \\ y_{n+1} \end{bmatrix} = \begin{bmatrix} x_n \\ y_n \end{bmatrix} + \gamma \nabla \log p_*(x_n, y_n) + \sqrt{2\gamma} V_n$$

where $V_n \sim \mathcal{N}\left(0, \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}\right)$.

Example 4.15 (Bayesian inference with ULA). Show that we can also straightforwardly perform Bayesian inference in this setting by derive ULA for a generic posterior density given $p(y|x)$ and $p(x)$.

Solution. Recall the target posterior density in this setting

$$\bar{p}_*(x|y) = p(x) \prod_{k=1}^n p(y_k|x),$$

where $p(x)$ is the prior density and $p(y_k|x)$ is the likelihood where observations are conditionally i.i.d given x . We can write the ULA iterates as

$$\begin{aligned} X_n &= X_{n-1} + \gamma \nabla \log \bar{p}_*(X_{n-1}|y) + \sqrt{2\gamma} V_n, \\ &= X_{n-1} + \gamma \left(\nabla \log p(X_{n-1}) + \sum_{k=1}^n \nabla \log p(y_k|X_{n-1}) \right) + \sqrt{2\gamma} V_n. \end{aligned}$$

Exercise 4.8. Let

$$p_*(x) = \frac{\Gamma(\frac{\nu+1}{2})}{\sqrt{\nu\pi} \Gamma(\frac{\nu}{2})} \left(1 + \frac{x^2}{\nu}\right)^{-(\nu+1)/2},$$

where $\nu > 0$ is a parameter. This is called *Student's t-distribution*. Derive the gradient of $\log p_*(x)$ and implement the unadjusted Langevin algorithm and Metropolis-adjusted Langevin algorithm for this distribution. Plot the histogram of the samples for different values of ν and γ and discuss any interesting result you observe.

A common problem arising in machine learning and statistics is *big data*, where the number of observations n is large. In this case, both ULA and MALA are infeasible as both require the iterates above to be evaluated, e.g., each iteration involves summing n terms. If n is order of millions, this is computationally infeasible. In this case, we can use the *stochastic* gradients. This is only applicable in the setting of ULA as we will see below, which is one reason why ULA-type methods are more popular than MALA-type methods.

4.5.1 STOCHASTIC GRADIENT LANGEVIN DYNAMICS

The problem of large number of data points arise in the setting of ULA as a sum, therefore, we should look for estimating large sums with something cheaper. Consider the following sum of (arbitrary) numbers:

$$g = \frac{1}{n} \sum_{i=1}^n g_i.$$

If n is simply too large to compute this sum efficiently, we can instead resort to *unbiased* estimates of it. This can be done by sampling $i_1, \dots, i_K \sim \{1, \dots, n\}$ uniformly and constructing

$$\hat{g} = \frac{1}{K} \sum_{k=1}^K g_{i_k}.$$

This estimate is an unbiased estimate of g , i.e.,

$$\mathbb{E}[\hat{g}] = \mathbb{E} \left[\frac{1}{K} \sum_{k=1}^K g_{i_k} \right] = \frac{1}{K} \sum_{k=1}^K \mathbb{E}[g_{i_k}] = g.$$

This idea can be used to construct stochastic gradients. We provide some examples below.

Example 4.16 (Large scale Bayesian inference). Recall the problem setting in Example 4.15:

$$X_t = X_{t-1} + \gamma \left(\nabla \log p(x) + \sum_{k=1}^n \nabla \log p(y_k | X_{t-1}) \right) + \sqrt{2\gamma} V_t.$$

Design the stochastic gradient sampler for this case.

Solution. Assume we sample uniformly $i_1^t, \dots, i_K^t \sim \{1, \dots, n\}$ at iteration t . We can then construct the following unbiased estimate of the sum:

$$\sum_{k=1}^n \nabla \log p(y_k | X_{t-1}) \approx \frac{n}{K} \sum_{k=1}^K \nabla \log p(y_{i_k^t} | X_{t-1}).$$

Note that (n/K) factor comes here as the sum itself did not have $(1/n)$ term (as opposed to the sum example above). Therefore, the stochastic gradient Langevin dynamics (SGLD) iterate can be written as

$$X_t = X_{t-1} + \gamma \left(\nabla \log p(x) + \frac{n}{K} \sum_{k=1}^K \nabla \log p(y_{i_k^t} | X_{t-1}) \right) + \sqrt{2\gamma} V_t.$$

This is also called *data subsampling* as one can see that the gradient only uses a subset of the data. Every iteration is cheap and computable as we only need to compute K terms. This is a very popular method in Bayesian inference and is used in many applications.

4.6 MONITORING AND POSTPROCESSING MCMC OUTPUT

There are a number of ways to monitor the MCMC samples to ensure that the algorithm is working as expected. We will discuss a few of them here.

4.6.1 TRACE PLOTS

The simplest way to monitor the MCMC output is to plot the trace of the samples. This is a plot of the samples against the iteration number. This is what we have been doing in previous examples. If the trace plots show you that the chain is still “moving”, then you can conclude that the chain is not yet converged. On the other hand, a trace plot from MH can also show you that the chain is stuck. It is then straightforward to conclude simple convergence issues from trace plots.

4.6.2 AUTOCORRELATION PLOTS

The autocorrelation plot is a plot of the autocorrelation function of the samples. The autocorrelation function is defined as

$$\rho_k = \frac{\text{Cov}(X_t, X_{t+k})}{\text{Var}(X_t)}.$$

This can be empirically computed on the samples coming from the Markov chain $(x_k)_{k \in \mathbb{N}}$. Since the aim of MCMC is to obtain nearly independent samples from a target p_* , we expect a good MCMC chain to exhibit low autocorrelation. A bad chain which is not *mixing* well will exhibit high autocorrelation. An example can be seen from Fig. 4.6 and see its caption for more details. One way to choose the proposal variance is to ensure that the chain has a low autocorrelation. This is a very simple way to monitor the chain.

4.6.3 EFFECTIVE SAMPLE SIZE

There is a notion of effective sample size for MCMC methods. However, its computation is trickier than the IS one and it is usually implemented using software packages. The definition of the ESS for MCMC chains is given as

$$\text{ESS} = \frac{N}{1 + 2 \sum_{k=1}^{\infty} \rho_k},$$

where ρ_k is the autocorrelation function. The ESS is an approximate measure of the number of independent samples that we have. For example, if the chain exhibits no autocorrelation, then the ESS is equal to the number of samples. If the chain exhibits high autocorrelation, then the ESS will be very low, as the sum in denominator will be large.

The computation of effective sample size in MCMC is usually done by software packages. We will not go into the details of this computation here.

4.6.4 THINNING THE MCMC OUTPUT

One way to reduce the autocorrelation of the MCMC samples is to *thin* them. This is a post-processing step that is done after the MCMC chain has been generated. The idea is to discard

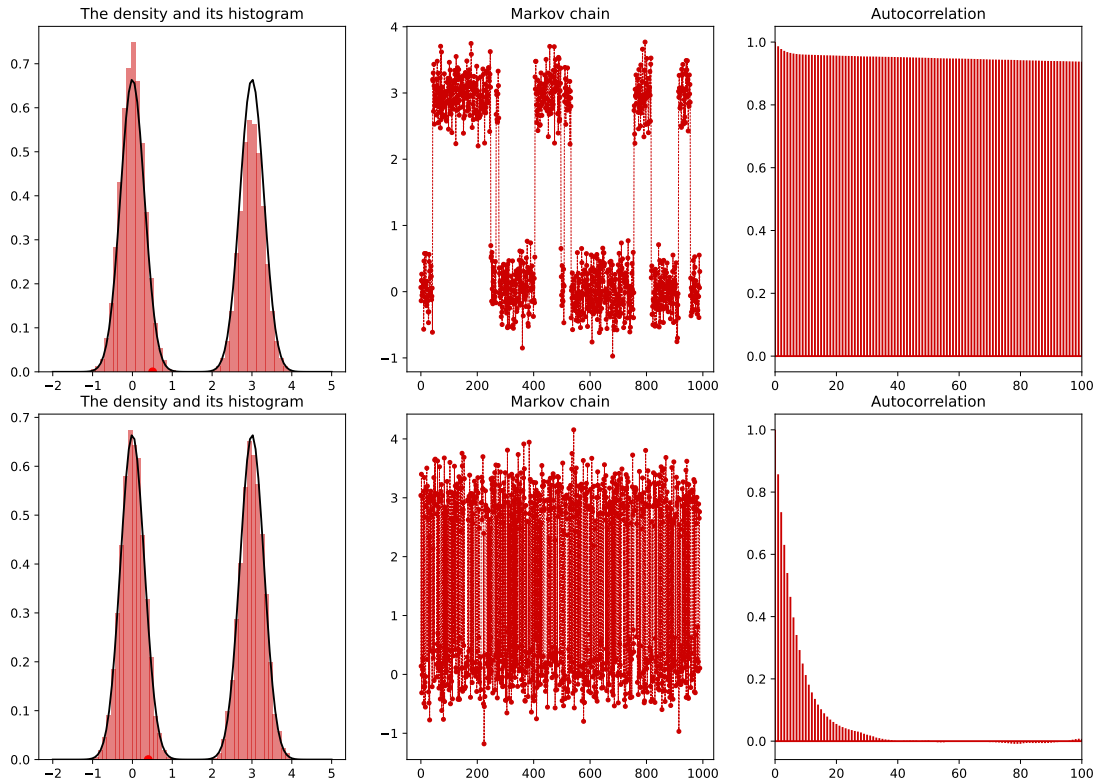


Figure 4.6: Random walk Metropolis-Hastings for a mixture of two Gaussians. The top panel shows the situation where $\sigma_q = 0.5$, so chain gets stuck in modes. This causes a high autocorrelation, and as such, this sampler is not considered to be a good one. When we set $\sigma_q = 4$, then the chain exhibits low autocorrelation and is a good sampler.

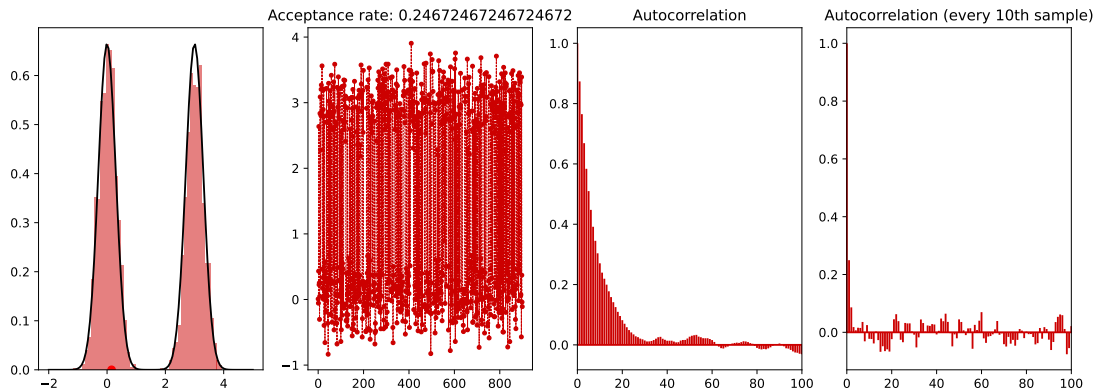


Figure 4.7: Thinning of MCMC samples. We keep every 10th sample for the same mixture of Gaussians example with $\sigma_q = 2$. It can be seen that the thinned MCMC chain exhibits significantly lower autocorrelation.

some of the samples and keep only a subset of them. This is done by keeping every k th sample. Since autocorrelation in an MCMC chain decays naturally over time, after reaching stationary, we can choose every k th of them and discard the rest. This will still give us a chain with the same stationary measure but with a lower autocorrelation. A demonstration of this can be seen from Fig .4.7.

4.7 MCMC FOR OPTIMISATION

4.7.1 AN INTRODUCTION TO OPTIMISATION

Optimisation is a fundamental task we face repeatedly in numerical computation, virtually all fields of engineering, machine learning, statistics, and so on. The task is to find the minima of a cost function $f : \mathbb{R}^d \rightarrow \mathbb{R}$. More formally, in this section, we will look at simulation-based methods to solve the problem of the form

$$x^* \in \operatorname{argmin}_{x \in X} f(x). \quad (4.10)$$

We will usually take $X = \mathbb{R}^d$, although simulation methods will adapt easily to the case where X could be a closed, bounded set. The typical algorithms to solve (4.10) are *gradient-based*. This means that the typical numerical way to solve the problem (4.10) is to construct a sequence $(x_t)_{t \geq 0}$ so that

$$x_{n+1} = x_n - \gamma \nabla f(x_n), \quad (4.11)$$

where $\gamma > 0$ is a step size. This is the *gradient descent* algorithm. Particularly relevant to this chapter is the fact that the gradient descent algorithm can be seen as a discretisation of a continuous-time ordinary differential equation (ODE) called the *gradient flow*:

$$\frac{dx_t}{dt} = -\nabla f(x_t), \quad (4.12)$$

which can also be written informally as

$$dx_t = -\nabla f(x_t) dt.$$

The gradient flow has the intriguing property of converging to the minimisers exponentially fast for convex functions f .

Exercise 4.9. Consider the function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ given by

$$f(x) = \|x\|^2.$$

1. Show that f has a unique global minimiser x^* and find it.
2. Show that the solution of the gradient flow (4.12) is given by

$$x_t = e^{-2t} x_0.$$

3. Consider the (discrete-time) gradient descent recursion (4.11) with step size $\gamma > 0$:

$$x_{n+1} = x_n - \gamma \nabla f(x_n), \quad n \geq 0. \quad (4.13)$$

Derive a closed-form expression for x_n in terms of x_0 .

4. Prove that $x_n \rightarrow x^*$ as $n \rightarrow \infty$ for every $x_0 \in \mathbb{R}^d$ if and only if $0 < \gamma < 1$. Moreover, show that the convergence is geometric and give an explicit rate.

The gradient descent algorithm is a simple and effective way to solve optimisation problems. However, it has its own limitations. For example, the gradient descent algorithm can get stuck in local minima, saddle points, or plateaus. In this section, we will see how we can use MCMC methods to solve the *global* optimisation problem (4.10) – that is, to find the global minima of the cost function f without getting stuck in local minima.

Exercise 4.10. [A toy nonconvex loss where gradient descent converges to a local minimum] Consider the function $f : \mathbb{R} \rightarrow \mathbb{R}$ defined by

$$f(x) = \begin{cases} (x+1)^2, & x \leq 1, \\ (x-2)^2 + 3, & x \geq 1. \end{cases} \quad (4.14)$$

1. Show that f has a unique global minimiser at $x^* = -1$ and a (strict) local minimiser at $x_{\text{loc}} = 2$.
2. Consider the discrete-time gradient descent recursion with step size $\gamma > 0$,

$$x_{n+1} = x_n - \gamma f'(x_n), \quad n \geq 0, \quad (4.15)$$

where f' is the derivative of the corresponding branch (assume $x_0 \neq 1$ so that the iterates never hit the kink). Prove that for any $\gamma \in (0, \frac{1}{2})$:

$$\text{if } x_0 > 1 \text{ then } x_n \rightarrow 2, \quad \text{and if } x_0 < 1 \text{ then } x_n \rightarrow -1.$$

Conclude that gradient descent can converge to a *local* minimum depending on the initialisation.

Randomness can help escape local minima and saddle points, and as such, MCMC methods can be a good candidate to solve optimisation problems. Indeed, certain MCMC methods were originally motivated by optimisation problems. These methods are a good candidate to solve challenging, nonconvex optimisation problems with multiple minima due to the intrinsic noise in the algorithms. In this section, we will briefly look at two MCMC methods that can be used for optimisation: (i) simulated annealing and (ii) Langevin MCMC.

Remark 4.4. It is important to note how a sampler can be used as an optimiser. Consider the target density

$$p_{\star}^{\beta}(x) \propto \exp(-\beta f(x)),$$

where $\beta > 0$ is a parameter. It is known in the literature that the density $p_{\star}^{\beta}(x)$ concentrates around the minima of f as $\beta \rightarrow \infty$ (Hwang, 1980). This connection between probability distributions and optimisation spurred the development of sampling methods for optimisation. In what follows, we describe two methods that exploit this connection.

Algorithm 13 Simulated Annealing

- 1: X_0 .
- 2: **for** $t = 1, 2, \dots$ **do**
- 3: $X' \sim q(x|X_{t-1})$ (symmetric proposal, e.g., random walk)
- 4: Set β_t (e.g. $\beta_t = \sqrt{1+t}$)
- 5: Accept X_t with probability

$$\min \left\{ 1, \frac{\bar{p}_\star^{\beta_t}(X')}{\bar{p}_\star^{\beta_t}(X_{t-1})} \right\}.$$

- 6: Otherwise set $X_t = X_{t-1}$.
 - 7: **end for**
-

4.7.2 SIMULATED ANNEALING

Consider now a *sequence* of target distributions defined as

$$p_\star^{\beta_t}(x) \propto \exp(-\beta_t f(x)),$$

where $\beta_t > 0$ is a sequence of increasing parameters. This algorithm *anneals* the target distribution so that $p_\star^{\beta_t}(x)$ becomes concentrated around the minima of f . Typically an increasing sequence of β_t 's are chosen, so that as $t \rightarrow \infty$, $\beta_t \rightarrow \infty$, making the target distribution concentrated on the minima. Drawing from previous chapter's MH algorithm, we can easily develop the simulated annealing (SA) algorithm as seen in Algorithm 13.

One can see that this simulated annealing method takes a special and intuitive case for optimisation. If we look at the acceptance ratio

$$\frac{\bar{p}_\star^{\beta_t}(X')}{\bar{p}_\star^{\beta_t}(X_{t-1})} = \frac{\exp(-\beta_t f(X'))}{\exp(-\beta_t f(X_{t-1}))} = \exp(\beta_t (f(X_{t-1}) - f(X'))),$$

we can see that the acceptance ratio is a function of the difference in the objective function values. If $f(X') \leq f(X_{t-1})$, this proposal will take higher values, possibly bigger than 1 depends on the improvement. If, however, $f(X') \geq f(X_{t-1})$, the acceptance ratio will be small as it should be. Scheduling of $(\beta_t)_{t \geq 0}$ is a design problem that depends on the specific cost function under consideration.

Example 4.17. Consider the following challenging cost function

$$f(x) = -(\cos(50x) + \sin(20x))^2 \exp(-5x^2), \quad x \in [-1, 1]. \quad (4.16)$$

This is a function with multiple local minima and is nonconvex. The function has one global minimum and we aim at finding it. Describe the simulated annealing method with a schedule $\beta_t = \sqrt{1+t}$.

Solution. We see that $\beta_t \rightarrow \infty$ as t grows. We use a random walk proposal with a standard deviation of $\sigma_q = 0.1$. We implement this on the log domain. We initialise $X_0 \sim \text{Unif}(-1, 1)$. The algorithm is implemented as, given X_{t-1}

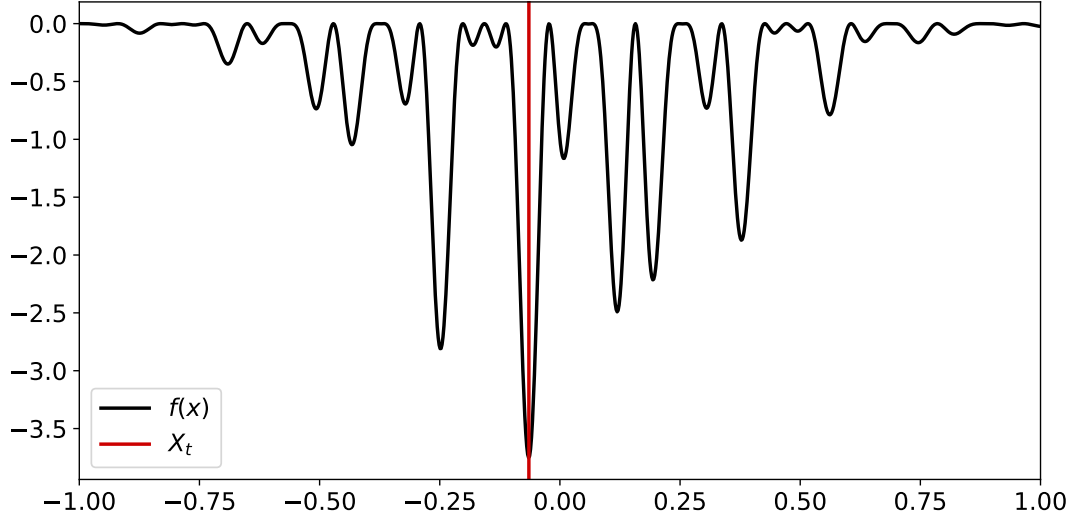


Figure 4.8: Simulated annealing for the function in Eq. 4.16. The red line shows the final estimate of SA algorithm.

- $X' \sim q(x|X_{t-1}) = \mathcal{N}(x; X_{t-1}, \sigma_q^2)$
- Sample $u \sim \text{Unif}(0, 1)$
- Accept if

$$\log(u) < \beta_t(f(X_{t-1}) - f(X'))$$

- Otherwise set $X_t = X_{t-1}$.

The result can be seen from Figure 4.8. We can see that the algorithm is able to find the global minima.

4.7.3 LANGEVIN MCMC FOR OPTIMISATION

In this section, we will describe how the Langevin MCMC based methods can be designed and used for optimisation. This class of methods are very popular in machine learning literature, both as a global optimiser and as a model of stochastic gradient descent (SGD) which is a popular optimisation algorithm in machine learning.

The use Langevin MCMC for optimisation is motivated by the fact that the following Langevin SDE

$$dX_t = -\nabla f(X_t)dt + \sqrt{\frac{2}{\beta}}dW_t, \quad (4.17)$$

leaves the target distribution $p_\star(x) \propto \exp(-\beta f(x))$ invariant. We have already seen that this sort of target concentrates around the minima of f as $\beta \rightarrow \infty$, therefore we can use this as an

optimisation algorithm. A simple Euler discretisation of this SDE gives

$$X_{n+1} = X_n - \gamma \nabla f(X_n) + \sqrt{\frac{2\gamma}{\beta}} W_n, \quad (4.18)$$

where $W_n \sim \mathcal{N}(0, I)$ and γ is the step size. This algorithm can be shown to converge to the global minima of f arbitrarily close for large β and small γ (Zhang et al., 2023).

The family of MCMC methods can be used for optimisation as well. We will showcase one example.

Example 4.18 (ULA for optimisation with a nonconvex potential). We reuse the nonconvex loss from Exercise 4.10:

$$f(x) = \begin{cases} (x+1)^2, & x \leq 1, \\ (x-2)^2 + 3, & x \geq 1. \end{cases}$$

Recall that this function has a unique global minimiser at $x^* = -1$ with $f(x^*) = 0$, and a strict local minimiser at $x_{\text{loc}} = 2$ with $f(x_{\text{loc}}) = 3$. Now consider the target distribution $p_\star^\beta \propto \exp(-\beta f)$ for $\beta > 0$. Show that the relative mass of $p_\star^\beta$ around the local minimiser 2 vanishes as $\beta \rightarrow \infty$. Then, describe how to use the Langevin discretisation targeting $p_\star^\beta$ to find the global minimiser.

Solution. As in simulated annealing, we turn optimisation into sampling by defining, for $\beta > 0$,

$$p_\star^\beta(x) \propto e^{-\beta f(x)}.$$

Differently from simulated annealing, we have a fixed β here. In this example the (unnormalised) density is explicitly piecewise Gaussian:

$$\bar{p}_\beta(x) = \begin{cases} \exp(-\beta(x+1)^2), & x \leq 1, \\ \exp(-\beta((x-2)^2 + 3)) = e^{-3\beta} \exp(-\beta(x-2)^2), & x \geq 1. \end{cases}$$

Therefore $p_\star^\beta$ is a “double-well” target with two modes (near -1 and 2), but the right mode is down-weighted by the factor $e^{-3\beta}$.

Fix a small $\varepsilon \in (0, 1)$. Consider two small neighbourhoods around the global and local minimisers, respectively:

$$\begin{aligned} B_- &:= B_\varepsilon(-1) = [-1 - \varepsilon, -1 + \varepsilon] \subset (-\infty, 1), \\ B_+ &:= B_\varepsilon(2) = [2 - \varepsilon, 2 + \varepsilon] \subset (1, \infty). \end{aligned}$$

Using the form of $\bar{p}_\beta$ and the change of variables $u = x + 1$ on B_- and $v = x - 2$ on B_+ , we obtain

$$\begin{aligned} \int_{B_-} \bar{p}_\beta(x) dx &= \int_{-\varepsilon}^{\varepsilon} e^{-\beta u^2} du, \\ \int_{B_+} \bar{p}_\beta(x) dx &= e^{-3\beta} \int_{-\varepsilon}^{\varepsilon} e^{-\beta v^2} dv. \end{aligned}$$

Hence

$$\frac{\int_{B_-} \bar{p}_\beta(x) dx}{\int_{B_+} \bar{p}_\beta(x) dx} = e^{3\beta}.$$

In particular,

$$\int_{B_+} p_\star^\beta(x) dx = \frac{\int_{B_+} \bar{p}_\beta(x) dx}{\int_{\mathbb{R}} \bar{p}_\beta(x) dx} \leq \frac{\int_{B_+} \bar{p}_\beta(x) dx}{\int_{B_-} \bar{p}_\beta(x) dx} = e^{-3\beta} \xrightarrow{\beta \rightarrow \infty} 0,$$

so the probability mass near the *local* minimiser 2 vanishes exponentially fast as $\beta \rightarrow \infty$. More generally, one can show that $p_\star^\beta \Rightarrow \delta_{-1}$ as $\beta \rightarrow \infty$ (but we will not as it is more involved).

To (approximately) sample from $p_\star^\beta$ without knowing its normalising constant, we can use the Langevin discretisation targeting $p_\star^\beta \propto e^{-\beta f}$, which is given by

$$X_{n+1} = X_n - \gamma f'(X_n) + \sqrt{\frac{2\gamma}{\beta}} W_n, \quad W_n \sim \mathcal{N}(0, 1), \quad (4.19)$$

where, away from the kink at $x = 1$,

$$f'(x) = \begin{cases} 2(x+1), & x < 1, \\ 2(x-2), & x > 1. \end{cases}$$

For fixed β and small γ , the chain in (4.19) has an invariant distribution close to $p_\star^\beta$ (the exact invariant law differs from $p_\star^\beta$ due to discretisation bias, as discussed earlier for the Gaussian case). To use this for optimisation, one lets $\beta = \beta_n \rightarrow \infty$ (cooling), so that the targets concentrate on the global minimiser -1 .

Exercise 4.11. Consider the following unadjusted Langevin algorithm:

$$X_{n+1} = X_n + \gamma \nabla \log p_\star(X_n) + \sqrt{2\gamma\beta^{-1}} W_{n+1},$$

where $W_{n+1} \sim \mathcal{N}(0, 1)$ and $\gamma > 0$ is the step size. Assume that $p_\star$ is a Gaussian:

$$p_\star(x) = \mathcal{N}(x; \mu, \sigma^2),$$

where $\mu \in \mathbb{R}^d$ and $\sigma^2 > 0$. Derive the limiting distribution $p_\star^{\gamma, \beta}(x)$ of X_n as $n \rightarrow \infty$. Discuss any constraints on γ for convergence and the limit $\beta \rightarrow \infty$.

BIBLIOGRAPHY

Agapiou, Sergios; Papaspiliopoulos, Omiros; Sanz-Alonso, Daniel; and Stuart, Andrew M. 2017. *Importance sampling: Intrinsic dimension and computational cost*. In *Statistical Science*, pp. 405–431. Cited on pp. 85 and 86.

Akyildiz, Omer Deniz. March 2019. *Sequential and adaptive Bayesian computation for inference and optimization*. Ph.D. thesis, Universidad Carlos III de Madrid. Can be accessed from: <http://akyildiz.me/works/thesis.pdf>. Cited on pp. 63, 67, and 75.

Akyildiz, Ömer Deniz and Míguez, Joaquín. 2021. *Convergence rates for optimised adaptive importance samplers*. In *Statistics and Computing*, vol. 31, no. 2, pp. 1–17. Cited on pp. 78 and 86.

Branchini, Nicola. 2024. *Simple Monte Carlo is independent of dimension. Or is it?* https://branchini.fun/posts/monte_carlo_indep/. Blog post. Cited on p. 65.

Cemgil, A Taylan. 2014. *A tutorial introduction to Monte Carlo methods, Markov Chain Monte Carlo and particle filtering*. In *Academic Press Library in Signal Processing*, vol. 1, pp. 1065–1114. Cited on p. 114.

Devroye, Luc. 1986. *Non-Uniform Random Variate Generation*. Cited on p. 17.

Douc, Randal; Moulines, Eric; Priouret, Pierre; and Soulier, Philippe. 2018. *Markov chains*. Springer. Cited on pp. 103, 106, and 109.

Douc, Randal; Moulines, Éric; and Stoffer, David. 2013. *Nonlinear Time Series: Theory, Methods and Applications with R Examples*. Chapman & Hall. Cited on p. 103.

Elvira, Víctor; Martino, Luca; and Robert, Christian P. 2018. *Rethinking the effective sample size*. In *International Statistical Review*. Cited on p. 95.

Hwang, Chii-Ruey. 1980. *Laplace's method revisited: weak convergence of probability measures*. In *The Annals of Probability*, pp. 1177–1182. Cited on p. 133.

Kallenberg, Olav. 2021. *Foundations of Modern Probability*. Probability Theory and Stochastic Modelling. Springer Cham, 3rd edn. ISBN 978-3-030-61871-1. doi:<https://doi.org/10.1007/978-3-030-61871-1>. 1 b/w illustration. Cited on p. 30.

Martino, Luca; Luengo, David; and Míguez, Joaquín. 2018. *Independent random sampling methods*. Springer. Cited on pp. 18, 34, 35, and 39.

Murphy, Kevin P. 2007. *Conjugate Bayesian analysis of the Gaussian distribution*. In def, vol. 1, no. $2\sigma^2$, p. 16. Cited on p. 6.

Owen, Art B. 2013. *Monte Carlo theory, methods and examples*. Cited on p. 95.

Robert, Christian P and Casella, George. 2004. *Monte Carlo statistical methods*. Springer. Cited on pp. 63 and 94.

———. 2010. *Introducing Monte Carlo methods with R*, vol. 18. Springer. Cited on p. 76.

Walters, Peter. 2000. *An introduction to ergodic theory*, vol. 79. Springer Science & Business Media. Cited on p. 19.

Yıldırım, Sinan. 2017. *Sabanci University IE 58001 Lecture notes: Simulation Methods for Statistical Inference*. Cited on pp. 48, 102, and 115.

Zhang, Ying; Akyildiz, Ömer Deniz; Damoulas, Theodoros; and Sabanis, Sotirios. 2023. *Nonasymptotic estimates for stochastic gradient Langevin dynamics under local conditions in nonconvex optimization*. In *Applied Mathematics & Optimization*, vol. 87, no. 2, p. 25. Cited on p. 136.